

RV1126B 音视频编解码& 推流开发手册

文档密级	仅供正点原子客户使用
------	------------



正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台: www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子天猫店铺 : <https://zhengdianyuanzi.tmall.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。



扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿:	邓志茂	正点原子	2026.02.04

目录

版权与使用协议	5
前言	6
第一章 RV1126B 视频处理单元介绍	8
1.1 编码解码参数	9
第二章 音视频编解码应用	10
2.1 摄像头编码	11
2.1.1 测试 MIPI 摄像头	11
2.1.2 摄像头编码成 H.264	12
2.1.3 摄像头编码成 H.265	13
2.2 摄像头 4K 显示	14
2.3 视频解码播放 4K	14
2.3.1 直接播放 4K 视频	15
2.3.2 居中播放 4K 视频	15
2.3.3 全屏播放 4K 视频	16
2.4 音频编解码	17
2.4.1 查看 GStreamer 音频解码插件	17
2.4.2 音频解码	18
2.4.3 查看 GStreamer 音频编码插件	18
2.4.4 音频编码	19
2.4.5 设置录音增益	20
2.4.6 编码录制 MIC 音频	20
第三章 音视频推流开发	22
3.1 Live555 流媒体软件	23
3.1.1 帮助说明	23
3.1.2 live555 推流	23
3.2 MediaMTX 流媒体软件	25
3.2.1 MediaMTX 简介	25
3.2.2 MediaMTX 配置文件	26
3.2.3 运行 MediaMTX	27
3.2.4 视频 RTSP 推流	27
3.2.5 视频 RTMP 推流	30
3.2.6 音视频 RTMP 推流	31
3.2.7 音视频 RTSP 推流	33
3.2.8 WebRTC 音视频	35
第四章 GStreamer 的音视频推流	39
4.1 GStreamer 音视频 RTSP 推流	40
4.1.1 程序源码及编译	40
4.1.2 帮助说明	41
4.1.3 设置录音增益	41
4.1.4 RTSP 推流	41
4.1.5 RTSP 拉流	42

版权与使用协议

致正点原子用户:

感谢您选择正点原子产品。为保障您的长期权益,我们为本产品配套开发了详尽的较高价值技术文档。这些文档凝聚了我们研发团队的大量心血与时间,是产品体验的重要组成部分。

为保护这份独特价值,防止其被无序扩散和复制,我们恳请您:

仅将本资料用于个人/团队学习,切勿公开上传至网络或分享给非购买用户。

您的配合是对正点原子持续投入原创文档内容的最大支持。谢谢!

前言

本文档介绍 RV1126B 的多媒体功能实现方案。RV1126B 搭载了由瑞芯微适配的硬件编解码器，并提供了两种主流的开发框架：GStreamer 与 ROCKIT。

GStreamer 作为成熟的通用多媒体框架，在 RV1126B 上表现稳定且易于使用，其代码可便捷地迁移至 RK3568、RK3588 等同系平台，实现高度复用。而 ROCKIT 作为瑞芯微自研的底层框架，虽然提供了更高的灵活性与控制粒度，但开发复杂度也相应增加。经实测，RV1126B 上的 ROCKIT 框架虽然能运行，但其配套文档更新滞后，部分 API 接口与实现存在不一致的情况，并且 ROCKIT 开发需要使用 IPC SDK。

在绝大多数应用场景中，GStreamer 框架已完全满足需求，且执行效率优异。综合评估，推荐使用 GStreamer 通用多媒体框架进行开发。

更多 RV1126B 视频关注 B 站 Up 主: dengzhimao

免责声明

本文档所提及的产品规格和使用说明仅供参考, 如有内容更新, 恕不另行通知; 除非有特殊约定, 本文档仅作为产品指导, 所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有, 未经公司的书面许可, 任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息, 请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

仅供正点原子客户使用

第一章 RV1126B 视频处理单元介绍

本文档介绍 RV1126B 型号的视频处理单元 (Video Processing Unit, VPU) 架构与功能。该 VPU 是 Rockchip 自主研发的专用视频处理硬件模块, 集成多种视频编解码和处理能力, 为嵌入式视觉应用提供高效的硬件加速支持。

1.1 编码解码参数

根据网盘资料 07、RK 官方文档\cn\RV1126B\Datasheet\Rockchip_RV1126B_Datasheet_V1.2-20250522.pdf 得知 RK 的编码解码参数如上。由上可知 RV1126B 支持 H.264 和 H.265 编码和解码，还支持并行 H.265+JPEG 和 H.264+JPEG 编码等参数。如下图。

1.2.6 Video CODEC

- Video Decoder
 - Real-time decoding of H.264 and H.265
 - Support MMU
 - H.265 HEVC/MVC Main Profile yuv420@L5.0 up to 3840x2160@30fps
 - H.264 AVC/MVC Main Profile yuv400/yuv420/yuv422@L5.1 up to 3840x2160@30fps
- Video Encoder
 - HEVC Main Profile, Level 5.0 High Tier
 - H.264 High Profile, Level 5.0
 - JPEG Baseline
 - Parallel encoding (HEVC+JPEG or H264+JPEG)
 - Support up to 12M@30fps
 - Support common primary stream such as 3840x2160@30fps, 1920x1080@30fps
 - Bitrate up to 200Mbps with CBR/VBR/FixQP/QPMAP bitrate control
 - YUV420 and YUV400 stream format

Copyright 2025 © Rockchip Electronics Co., Ltd.

10

所以它只能处理 H.264/H.265 的编码视频，我们就不要播放如 MPEG、Vp8 等编码的视频了，RV1126B 是播放不了的！

如果不会看视频格式请使用格式工厂 Windows 软件看，或者在开发板使用 ffprobe 指令查看视频编码格式。如下图，使用 ffprobe 指令就可以查看编码格式，可以看到编码为 h264，帧率 30 等等信息。

```
ffprobe xxx.mp4
```

[illegible]

第二章 音视频编解码应用

本章介绍如何在 RV1126B 上测试编解码，及开发相关应用。

这里写了常规的一些编码应用，更详细的请看 GStreamer 视频编解码适配方法可以参考网盘资料 07、RK 官方文档\cn\Linux\Multimedia\Rockchip_User_Guide_Linux_Gstreamer_CN.pdf。

仅供正点原子客户使用

2.1 摄像头编码

本小节旨在完成摄像头功能测试,具体包括验证摄像头设备可用性及视频数据编码两部分。本文档聚焦于 MIPI 摄像头方案,其通过专用接口直接传输 NV12 格式的原始视频流。

若使用 USB 摄像头,需注意其通常以 YUY2 格式采集数据,且受限于 USB 带宽,分辨率与帧率呈负相关——分辨率越高,支持帧率越低。这也解释了为何在 4K 等高分辨率场景下推荐采用 MIPI 摄像头方案。

注:部分 USB 摄像头虽可通过 MJPG 格式支持 4K 采集,但此类方案均不在本文档讨论范围内。

2.1.1 测试 MIPI 摄像头

正点原子 RV1126B 支持 imx415 (4K) 和 imx335 (2K) 摄像头,店铺有售卖。推荐 imx415,因为它支持 4K30FPS,显示效果也较好。

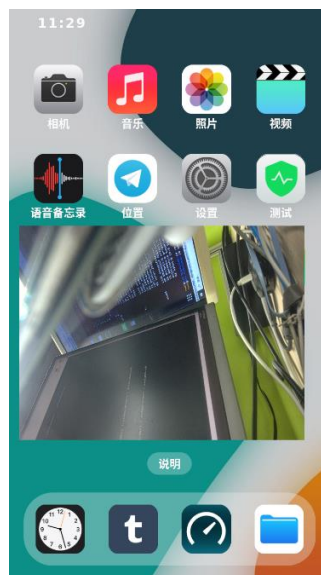
检查摄像头工作状态:连接 MIPI 摄像头后,确认系统是否已识别并创建 `/dev/video-camera0` 设备节点。

```
gst-launch-1.0 v4l2src device=/dev/video-camera0 ! \
video/x-raw,format=NV12,width=640,height=480,framerate=30/1 ! \
videoconvert ! \
waylandsink
```

如下图,这样表示摄像头能正常工作。

```
root@RK-OLRV1126B:/# gst-launch-1.0 v4l2src device=/dev/video-camera0 ! video/x-raw,format=NV12,width=640,height=480,framerate=30/1 ! videoconvert ! waylandsink
Setting pipeline to PAUSED ...
Using mplane plugin for capture
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
[ 374.528059] rkisp_hw 21d00000.isp: set isp clk = 500000000Hz
[ 374.549315] rkisp rkisp-vir0: first params buf queue
[ 374.549748] rkci1-mipi-lvds: stream[0] start streaming
[ 374.550051] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 00000000e1bcc296, sd_name:rockchip-csi2-dphy0
[ 374.550070] rockchip-mipi-csi2 mipi0-csi2: stream ON
[ 374.550110] rockchip-csi2-dphy0 dphy0, data_rate mbps 892
[ 374.550135] rockchip-csi2-dphy0 csi2-dphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[ 374.550145] imx415 3-001a: s_stream: 1, 3864x2192, hdr: 0, bpp: 10
[ 374.655126] rkci1-mipi-lvds: Warning: vblank need >= 1000us if isp work in online, cur 859 us
[ 374.688417] rkci1-mipi-lvds: Warning: vblank need >= 1000us if isp work in online, cur 859 us
Redistribute latency...
0:01:51.7 / 99:99:99.
```

MIPI 屏上也显示了摄像头画面,如下所示。



如果用户有两个 MIPI 摄像头,那么用下面的指令查询设备节点。

```
grep "/sys/class/video4linux/video*/name | grep mainpath
```

grep "/sys/class/video4linux/video*/name | grep rkisp_selfpath #子节点, 聪明的你可以想到 video23 与 video24 是同一个摄像头, 不同的出图设备节点, 可以一个用于录制视频, 一个用于显示 (预览) !

```
root@ATK-DLRV1126B:/# grep "/sys/class/video4linux/video*/name | grep mainpath
/sys/class/video4linux/video23/name:rkisp_mainpath
/sys/class/video4linux/video31/name:rkisp_mainpath
root@ATK-DLRV1126B:/# grep "/sys/class/video4linux/video*/name | grep rkisp_selfpath
/sys/class/video4linux/video24/name:rkisp_selfpath
/sys/class/video4linux/video32/name:rkisp_selfpath
root@ATK-DLRV1126B:/#
```

注上面的设备节点不会因为有没有插 MIPI 摄像头都会生成。所以如果用户有两 MIPI 摄像头时, 要测试另一个摄像头设备节点的话就要看现在对应的/dev/video-camera0 节点, 如下图。

```
ls /dev/video-camera0 -l
```

```
root@ATK-DLRV1126B:/# ls /dev/video-camera0 -l
lrwxrwxrwx 1 root root 7 Feb  2 11:08 /dev/video-camera0 -> video23
root@ATK-DLRV1126B:/#
```

现在的摄像头节点是 video23, 那么有两个 MIPI 摄像头时, 另一个 MIPI 摄像头的设备节点就是 video31!

所以测试另一个摄像头的指令为:

```
gst-launch-1.0 v4l2src device=/dev/video31 ! \
video/x-raw,format=NV12,width=640,height=480,framerate=30/1 ! \
videoconvert ! \
waylandsink
```

2.1.2 摄像头编码成 H.264

执行下面的指令将摄像头采集的数据编码成 H.264 文件。

```
gst-launch-1.0 v4l2src device=/dev/video-camera0 \
! video/x-raw,format=NV12,width=640,height=480, \
framerate=30/1 ! mpph264enc ! filesink location=test.h264
```

```
root@ATK-DLRV1126B:/# gst-launch-1.0 v4l2src device=/dev/video-camera0 \
> ! video/x-raw,format=NV12,width=640,height=480, \
> framerate=30/1 ! mpph264enc ! filesink location=test.h264
mpp[1591]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1591]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1591]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1591]: mpp: unable to create enc vp8 for soc rv1126b unsupported
mpp[1591]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
Setting pipeline to PAUSED ...
Using mplane plugin for capture
mpp[1591]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1591]: mpp: Only rk3588's h264/265/jpeg and rk3576's h264/265 encoder can use frame parallel
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
mpp[1591]: mpp_enc: MPP_ENC_SET_RC_CFG bps 1152000 [1080000 : 1224000] fps [30:30] gop 30
mpp[1591]: h264e api v2: MPP_ENC_SET_PREP_CFG w:h [640:480] stride [640:480]
mpp[1591]: mpp_enc: mode cbr_bps [1080000:1152000:1224000] fps fix [30/1] -> fix [30/1] gop i [30] v [0]
[ 1598.624783] rkisp_hw 21d00000.isp: set isp clk = 500000000Hz
[ 1598.647197] rkisp rkisp-vir0: first params buf queue
[ 1598.647617] rkicf-mipi-lvds: stream[0] start streaming
[ 1598.647904] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 00000000e1bcc296, sd_name:rockchip-csi2-dphy0
[ 1598.647919] rockchip-mipi-csi2 mipi0-csi2: stream ON
[ 1598.647957] rockchip-csi2-dphy0: dphy0, data_rate mbps 892
[ 1598.647982] rockchip-csi2-dphy csi2-dphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[ 1598.647992] imx415 3-001a: s_stream: 1. 3864x2192, hdr: 0, bpp: 10
[ 1598.754790] rkicf-mipi-lvds: Warning: vblank need >= 1000us if isp work in online, cur 859 us
[ 1598.788120] rkicf-mipi-lvds: Warning: vblank need >= 1000us if isp work in online, cur 859 us
Redistribute latency...
0:00:05.2 / 99:99:99.
```

按 Ctrl+C 就可以停止录制。然后查看当前路径下产生的 test.h264 文件。

```
root@ATK-DLRV1126B:/# ls test.h264
test.h264
root@ATK-DLRV1126B:/#
```

使用 gst-play-1.0 就可以播放此类型文件, 如下图。

gst-play-1.0 test.h264

```
root@ATK-DLRV1126B:/# gst-play-1.0 test.h264
Press 'k' to see a list of keyboard shortcuts.
Now playing /test.h264
Redistribute latency...
mpp[1623]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1623]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1623]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1623]: mpp: unable to create enc vp8 for soc rv1126b unsupported
mpp[1623]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1623]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1623]: h264d_api: is_avc=1
mpp[1623]: mpp_buf_slot: mismatch h_stride_by_pixel 704 - 640
mpp[1623]: mpp_buf_slot: mismatch h_stride_by_byte 704 - 640
mpp[1623]: mpp_buf_slot: mismatch size_total 506880 - 614400
mpp[1623]: mpp_buf_slot: mismatch h_stride_by_pixel 704 - 640
mpp[1623]: mpp_buf_slot: mismatch h_stride_by_byte 704 - 640
mpp[1623]: mpp_buf_slot: mismatch size_total 506880 - 614400
Redistribute latency...
0:00:03.2 / 0:00:21.6
root@ATK-DLRV1126B:/#
```

若还需要封装 MP4, 输入下面的指令。录制 5 秒 H.264 编码 mp4 视频。

```
gst-launch-1.0 v4l2src device=/dev/video-camera0 \
num-buffers=150 ! video/x-raw,format=NV12, \
width=640,height=480, framerate=30/1 ! mpph264enc \
! h264parse ! qtmux ! filesink location=testh264.mp4
```

或者不限时长。

```
gst-launch-1.0 -e v4l2src device=/dev/video-camera0 \
! video/x-raw,format=NV12, \
width=640,height=480, framerate=30/1 ! mpph264enc \
! h264parse ! qtmux ! filesink location=testh264.mp4
```

2.1.3 摄像头编码成 H.265

执行下面的指令将摄像头采集的数据编码成 H.265 文件。

```
gst-launch-1.0 v4l2src device=/dev/video-camera0 \
! video/x-raw,format=NV12,width=640,height=480, \
framerate=30/1 ! mpph265enc ! filesink location=test.h265
```

```
root@ATK-DLRV1126B:/# gst-launch-1.0 v4l2src device=/dev/video-camera0 ! video/x-raw,format=NV12,width=640,height=480, framerate=30/1 ! mpph265enc ! filesink location=test.h265
mpp[1639]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1639]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1639]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1639]: mpp: unable to create enc vp8 for soc rv1126b unsupported
mpp[1639]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
Setting pipeline to PAUSED ...
Using mplane plugin for capture
mpp[1639]: mpp_info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1639]: mpp: only rk3588's h264/265/jpeg and rk3578's h264/265 encoder can use frame parallel
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
mpp[1639]: mpp_enc: MPP_ENC_SET_RC_CFG bps 1152000 [1080000 : 1224000] fps [30:30] gop 30
mpp[1639]: h265e_api: h265e_proc_prep_cfg MPP_ENC_SET_PREP_CFG w:h [640:480] stride [640:480]
mpp[1639]: mpp_enc: mode cbr bps [1080000:1152000:1224000] fps fix [30/1] -> fix [30/1] gop i [30] v [0]
[ 2110.350777] rkisp_hw 21d00000.isp: set isp clk = 500000000Hz
[ 2110.362105] rkisp rkisp-vi0: first params buf queue
[ 2110.362465] rkcf-mipi-lvds: stream[0] start streaming
[ 2110.362747] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 00000000e1bcc296, sd_name:rockchip-csi2-dphy0
[ 2110.362761] rockchip-mipi-csi2 mipi0-csi2: stream ON
[ 2110.362800] rockchip-csi2-dphy0 dphy0: data rate mbps 892
[ 2110.362825] rockchip-csi2-dphy0 csi2-dphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[ 2110.362835] imx415 3-001a: s_stream: 1, 3864x2192, hdr: 0, bpp: 10
[ 2110.468881] rkcf-mipi-lvds: Warning: vblank need >= 1000us if isp work in online, cur 859 us
[ 2110.502215] rkcf-mipi-lvds: Warning: vblank need >= 1000us if isp work in online, cur 859 us
Redistribute latency...
```

按 Ctrl+C 就可以停止录制。然后查看当前路径下产生的 test.h265 文件。

```
root@ATK-DLRV1126B:/# ls test.h265
test.h265
root@ATK-DLRV1126B:/#
```

使用 gst-play-1.0 就可以播放此类型文件, 如下图。

```
gst-play-1.0 test.h265
```

```

root@ATK-DLRV1126B:/# gst-play-1.0 test.h265
Press 'k' to see a list of keyboard shortcuts.
Now playing /test.h265
Redistribute latency...
mpp[1652]: mpp_info: mpp version: 215fc18 author: alientek      2025-10-20 repository init!
mpp[1652]: mpp_info: mpp version: 215fc18 author: alientek      2025-10-20 repository init!
mpp[1652]: mpp_info: mpp version: 215fc18 author: alientek      2025-10-20 repository init!
mpp[1652]: mpp: unable to create enc vp8 for soc rv1126b unsupported
mpp[1652]: mpp_info: mpp version: 215fc18 author: alientek      2025-10-20 repository init!
mpp[1652]: mpp_info: mpp version: 215fc18 author: alientek      2025-10-20 repository init!
mpp[1652]: H265D_PARSER: extradata is encoded as hvcC format
mpp[1652]: mpp_buf_slot: mismatch h_stride_by_pixel 704 - 640
mpp[1652]: mpp_buf_slot: mismatch h_stride_by_byte 704 - 640
mpp[1652]: mpp_buf_slot: mismatch size_total 608256 - 552960
mpp[1652]: mpp_buf_slot: mismatch h_stride_by_pixel 704 - 640
mpp[1652]: mpp_buf_slot: mismatch h_stride_by_byte 704 - 640
mpp[1652]: mpp_buf_slot: mismatch size_total 608256 - 552960
Redistribute latency...
0:00:05.2 / 0:00:07.7
root@ATK-DLRV1126B:/#

```

若还需要封装 MP4, 输入下面的指令。录制 5 秒 H.265 编码 mp4 视频。

```

gst-launch-1.0 v4l2src device=/dev/video-camera0 \
num-buffers=150 ! video/x-raw,format=NV12, \
width=640,height=480, framerate=30/1 ! mpph265enc \
! h265parse ! qtmux ! filesink location=testh265.mp4

```

或者不限时长。

```

gst-launch-1.0 -e v4l2src device=/dev/video-camera0 \
! video/x-raw,format=NV12, \
width=640,height=480, framerate=30/1 ! mpph265enc \
! h265parse ! qtmux ! filesink location=testh265.mp4

```

2.2 摄像头 4K 显示

需要插上 imx415 摄像头, imx335 不支持 4K。

单路 4K 显示。

执行下面指令 4K 显示摄像头。由于摄像头是 4K (3840x2160), 画面特别大, 会覆盖整个屏幕(720x1280), 只能显示部分。可以在末尾加个 fullscreen=true, 这样就完全显示了。

```

gst-launch-1.0 v4l2src device=/dev/video-camera0 \
! video/x-raw,format=NV12,width=3840,height=2160,framerate=30/1 \
! waylandsink

```

双路 4K 显示。(需要插上两个 imx415 摄像头, imx335 不支持 4K。)

由于 ISP 最大像素处理为 12M, 一个 4K 摄像头 imx415 就已经达到了 8M, 所以 2 路 4K 不能同时都满跑 30FPS。

```

gst-launch-1.0 v4l2src device=/dev/video23 \
! video/x-raw,format=NV12,width=3840,height=2160 \
! videoconvert ! waylandsink sync=false \
v4l2src device=/dev/video31 ! \
video/x-raw,format=NV12,width=3840, \
height=2160 ! videoconvert ! waylandsink sync=false

```

2.3 视频解码播放 4K

GStreamer 瑞芯微已经适配硬件解码, 对于 RV1126B 来说, 视频解码只支持 H.264 和 H.265 解码, 其他编码不支持, 所以不要播放其它编码格式视频。

首先的系统需要有一个 4K30FPS 的 (4K60FPS 也只能解码成 30FPS) 视频文件, 请自行上传到开发板, H.264/H.265 编码, 如下图。

```
root@ATK-DLRV1126B:/# ls Santorini4K30FPS.mp4
Santorini4K30FPS.mp4
root@ATK-DLRV1126B:/#
```

请注意: 播放前请用 `alsamixer` 调小播放音量! 默认播放音量是最大的, 喇叭可能容易破音。

2.3.1 直接播放 4K 视频

直接用 `gst-play-1.0` 播放。但是这样播放的视频画面非常大, 覆盖了整个屏幕!

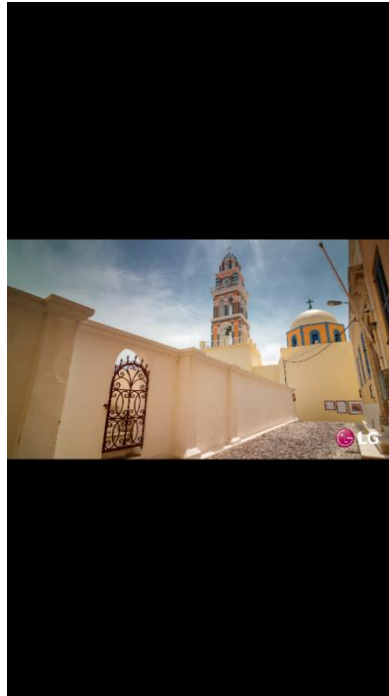
```
gst-play-1.0 Santorini4K30FPS.mp4
```



2.3.2 居中播放 4K 视频

完整居中显示, 使用下面的指令播放。

```
gst-launch-1.0 playbin uri=file://$(pwd)/Santorini4K30FPS.mp4 \
video-sink="queue ! h264parse ! mppvideodec ! videoconvert ! waylandsink fullscreen=true" \
audio-sink="autoaudiosink"
```



2.3.3 全屏播放 4K 视频

由于视频是横屏的，而屏幕是竖屏的，可以使用下面指令全屏播放视频。

```
gst-launch-1.0 playbin uri=file://$(pwd)/Santorini4K30FPS.mp4 \  
video-sink="videoconvert ! videoflip method=clockwise ! waylandsink fullscreen=true"
```



2.4 音频编解码

在 Rockchip RV1126B 平台上, 音频确实没有独立的硬件解码器, 音频编解码主要依赖 CPU 软件处理。处理音频对于 RV1126B 的 CPU 来说, 音频编码计算量相对小 (每秒几万个样本 $\times$ 较简单算法), 四核心 A53 对于编解码音频来说, 占用率仅%2 左右, 要知道 4 核心 A53 可达 400%。基本不怎么占 CPU!

2.4.1 查看 GStreamer 音频解码插件

执行下面指令查看 GStreamer 的音频解码插件。若查看出来的插件不一样, 请更新到最新系统。

```
gst-inspect-1.0 | grep -E "decode.*audio|audio.*decode" -i
```

```
root@ATK-DLVR1126B:~# gst-inspect-1.0 | grep -E "decode.*audio|audio.*decode" -i
dvdpcmdec: dvdpcmdec: DVD LPCM Audio decoder
faad: faad: AAC audio decoder
libav: avdec_aac: libav AAC (Advanced Audio Coding) decoder
libav: avdec_aac_fixed: libav AAC (Advanced Audio Coding) decoder
libav: avdec_aac_latm: libav AAC LATM (Advanced Audio Coding LATM syntax) decoder
libav: avdec_adpcm_vima: libav LucasArts VIMA audio decoder
libav: avdec_alac: libav ALAC (Apple Lossless Audio Codec) decoder
libav: avdec_als: libav MPEG-4 Audio Lossless Coding (ALS) decoder
libav: avdec_ape: libav Monkey's Audio decoder
libav: avdec_aptx: libav aptX (Audio Processing Technology for Bluetooth) decoder
libav: avdec_aptx_hd: libav aptX HD (Audio Processing Technology for Bluetooth) decoder
libav: avdec_avs: libav AVS (Audio Video Standard) video decoder
libav: avdec_binkaudio_dct: libav Bink Audio (DCT) decoder
libav: avdec_binkaudio_rdft: libav Bink Audio (RDFT) decoder
libav: avdec_bmv_audio: libav Discworld II BMV audio decoder
libav: avdec_cavs: libav Chinese AVS (Audio Video Standard) (AVS1-P2, JiZhun profile) decoder
libav: avdec_cook: libav Cook / Cooker / Gecko (RealAudio G2) decoder
libav: avdec_dsicinaudio: libav Delphine Software International CIN audio decoder
libav: avdec_dvaudio: libav Ulead DV Audio decoder
libav: avdec_fastaudio: libav MobiClip FastAudio decoder
libav: avdec_flac: libav FLAC (Free Lossless Audio Codec) decoder
libav: avdec_hcom: libav HCOM Audio decoder
libav: avdec_iac: libav IAC (Indeo Audio Coder) decoder
libav: avdec_mace3: libav MACE (Macintosh Audio Compression/Expansion) 3:1 decoder
libav: avdec_mace6: libav MACE (Macintosh Audio Compression/Expansion) 6:1 decoder
libav: avdec_mp1float: libav MP1 (MPEG audio layer 1) decoder
libav: avdec_mp2float: libav MP2 (MPEG audio layer 2) decoder
libav: avdec_mp3: libav MP3 (MPEG audio layer 3) decoder
libav: avdec_mp3adu: libav ADU (Application Data Unit) MP3 (MPEG audio layer 3) decoder
libav: avdec_mp3adufloat: libav ADU (Application Data Unit) MP3 (MPEG audio layer 3) decoder
libav: avdec_mp3float: libav MP3 (MPEG audio layer 3) decoder
libav: avdec_on2avc: libav On2 Audio for Video Codec decoder
libav: avdec_paf_audio: libav Amazing Studio Packed Animation File Audio decoder
libav: avdec_ralf: libav RealAudio Lossless decoder
libav: avdec_real_144: libav RealAudio 1.0 (14.4K) decoder
libav: avdec_real_288: libav RealAudio 2.0 (28.8K) decoder
libav: avdec_sipr: libav RealAudio SIPR / ACELP.NET decoder
libav: avdec_smackaud: libav Smacker audio decoder
libav: avdec_tak: libav TAK (Tom's lossless Audio Kompressor) decoder
libav: avdec_tta: libav TTA (True Audio) decoder
libav: avdec_vmdaudio: libav Sierra VMD audio decoder
libav: avdec_wmalossless: libav Windows Media Audio Lossless decoder
libav: avdec_wmapro: libav Windows Media Audio 9 Professional decoder
libav: avdec_wmav1: libav Windows Media Audio 1 decoder
libav: avdec_wmav2: libav Windows Media Audio 2 decoder
libav: avdec_wmavoice: libav Windows Media Audio Voice decoder
libav: avdec_ws_snd1: libav Westwood Audio (SND1) decoder
libav: avdec_xma1: libav Xbox Media Audio 1 decoder
libav: avdec_xma2: libav Xbox Media Audio 2 decoder
mpg123: mpg123audiodec: mpg123 mp3 decoder
opus: opusdec: Opus audio decoder
rtsp: rtspreddec: Redundant Audio Data (RED) Decoder
vorbis: vorbisdec: Vorbis audio decoder
root@ATK-DLVR1126B:~#
```

可以看到上面非常多音频插件, 其中大部分是 **libav**, **libav** 是一个跨平台的多媒体处理库, 它是 FFmpeg 项目的一个分支。在 GStreamer 环境中, **libav** 提供了大量的编解码器插件。

其中常用的音频解码器如下:

- **AAC 相关 (现代设备最常用):**
 - faad - 专门的 AAC 解码器
 - avdec_aac - libav 的 AAC 解码器
- **MP3 相关:**
 - avdec_mp3 - MP3 解码器
 - mpg123 - 另一个 MP3 解码器
- **开源格式:**

opusdec - Opus (WebRTC、VoIP 常用)
vorbisdec - Vorbis (OGG 容器常用)
avdec_flac - FLAC 无损音频

- **Windows 媒体格式:**

avdec_wmv1/wmv2 - WMA 解码器

2.4.2 音频解码

执行下面的指令解码音频, 随意找一个路径下的 mp3 文件, 如果你的系统没有 mp3 文件, 请自行上传到开发板。

gst-play-1.0 /oem/piano2-CoolEdit.mp3 # 一般会自动选择解码器播放, 我们无需要指定
可以使用 ffprobe 或者 gst-discoverer-1.0 查看 mp3 的编码。

gst-discoverer-1.0 -v /oem/piano2-CoolEdit.mp3

ffprobe /oem/piano2-CoolEdit.mp3 #或者用 ffprobe 查看此文件的编码。

```
root@ATK-DLRV1126B:/# gst-discoverer-1.0 -v /oem/piano2-CoolEdit.mp3
Analyzing file:///oem/piano2-CoolEdit.mp3
Done discovering file:///oem/piano2-CoolEdit.mp3

Properties:
  Duration: 0:00:06.360000000
  Seekable: yes
  Live: no
  Tags:
    has crc: true
    channel mode: joint-stereo
    audio codec: MPEG-1 Layer 3 (MP3)
    nominal bitrate: 128000
  audio #0: audio/mpeg, mpegversion=(int)1, mpegaudioversion=(int)1, layer=(int)3, rate=(int)48000, channels=(int)2, parsed=(boolean)true
  Tags:
    has crc: true
    channel mode: joint-stereo
    audio codec: MPEG-1 Layer 3 (MP3)
    nominal bitrate: 128000

Codec:
  audio/mpeg, mpegversion=(int)1, mpegaudioversion=(int)1, layer=(int)3, rate=(int)48000, channels=(int)2, parsed=(boolean)true
  Stream ID: f85c4fa7f32aad8baa7e2e1b7f7926427a4c0d87c0756bdf3ca782eb6745
  Language: <unknown>
  Channels: 2 (front-left, front-right)
  Sample rate: 48000
  Depth: 16
  Bitrate: 128000
  Max bitrate: 0

root@ATK-DLRV1126B:/#
```

由上图可知, 音频编码是 mp3。

2.4.3 查看 GSTreamer 音频编码插件

执行下面指令查看 GSTreamer 的音频编码插件。若查看出来的插件不一样, 请更新到最新系统。

gst-inspect-1.0 | grep -E "audio.*encoder|encoder.*audio|mp3.*encoder" -i

```
root@ATK-DLRV1126B:/# gst-inspect-1.0 | grep -E "audio.*encoder|encoder.*audio|mp3.*encoder" -i
lame: lamemp3enc: L.A.M.E. mp3 encoder
libav: lavenc_aac: libav AAC (Advanced Audio Coding) encoder
libav: lavenc_alac: libav ALAC (Apple Lossless Audio Codec) encoder
libav: lavenc_aptx: libav aptX (Audio Processing Technology for Bluetooth) encoder
libav: lavenc_aptx_hd: libav aptX HD (Audio Processing Technology for Bluetooth) encoder
libav: lavenc_mp2: libav MP2 (MPEG audio layer 2) encoder
libav: lavenc_mp2fixed: libav MP2 fixed point (MPEG audio layer 2) encoder
libav: lavenc_real_144: libav RealAudio 1.0 (14.4K) encoder
libav: lavenc_tta: libav TTA (True Audio) encoder
libav: lavenc_wmv1: libav Windows Media Audio 1 encoder
libav: lavenc_wmv2: libav Windows Media Audio 2 encoder
opus: opusenc: Opus audio encoder
rtp: rtpredenc: Redundant Audio Data (RED) Encoder
vorbis: vorbisenc: Vorbis audio encoder

root@ATK-DLRV1126B:/#
```

插件解释:

- **opusenc** - WebRTC 实时通信标准, 低延迟 (最常用)
- **lamemp3enc** - 使用 LAME (LAME Ain't an MP3 Encoder) 库编码 MP3 音频 (常用)

- **avenc_aac** - 移动设备、在线视频通用（最常用）
- **avenc_mp2** - 广播、电视音频
- **vorbisenc** - OGG 容器、开源项目（常用）
- **avenc_alac** - 苹果无损音频（常用）
- **avenc aptx** - 蓝牙高质量音频
- **avenc_wmav1/wmav2** - Windows 媒体（已过时）
- **avenc_real_144** - RealAudio（过时）

2.4.4 音频编码

执行下面的指令进行 opusenc 音频编码，按 Ctrl + C 结束。

```
gst-launch-1.0 -e audiotestsrc wave=sine freq=440 \
! audioconvert ! opusenc bitrate=64000 ! \
oggmux ! filesink location=test.ogg
```

- **gst-launch-1.0 -e** - GStreamer 命令行工具，-e 参数在流结束时发送 EOS（End Of Stream）事件
- **audiotestsrc wave=sine freq=440** - 音频测试源，生成 440Hz 正弦波
- **audioconvert** - 音频格式转换器，确保音频格式兼容
- **opusenc bitrate=64000** - Opus 音频编码器，以 64kbps 比特率编码
- **oggmux** - OGG 容器复用器，将音频流封装到 OGG 容器
- **filesink location=test.ogg** - 文件输出，保存为 test.ogg 文件

执行下面指令编码 mp3，按 Ctrl + C 停止。

```
gst-launch-1.0 -e audiotestsrc wave=sine freq=440 ! \
audioconvert ! lamemp3enc bitrate=128 ! \
filesink location=test.mp3
```

指令解释：

- **gst-launch-1.0** - GStreamer 命令行工具，用于构建和运行多媒体管道。
- **-e** - 命令行参数，在接收到流结束信号时发送 EOS 事件，确保文件正确结束。
- **audiotestsrc** - 音频测试源元素，生成测试音频信号。
- **wave=sine freq=440** - 音频测试源的参数，设置波形为正弦波，频率为 440Hz。
- **audioconvert** - 音频格式转换元素，确保不同音频元素间的格式兼容性。
- **lamemp3enc** - LAME MP3 编码器元素，将原始音频编码为 MP3 格式。
- **bitrate=128** - MP3 编码器参数，设置编码比特率为 128kbps。
- **filesink** - 文件输出元素，将数据流写入文件。
- **location=test.mp3** - 文件输出参数，指定输出文件名为 test.mp3。

2.4.5 设置录音增益

开发板板载的麦克风（咪头）录音增益默认设置较小，为确保录音音量充足，需要先执行以下指令将录音增益调整到合适的较大值。

```
/usr/bin/amixer -c 0 cset name='ADC OSR Volume ON' 'on'
/usr/bin/amixer -c 0 set 'ADC OSR' 100%
/usr/bin/amixer -c 0 set 'ADC2DAC Mixer' 90%
```

```
root@ATK-DLRV1126B:~# /usr/bin/amixer -c 0 cset name='ADC OSR Volume ON' 'on'
numid=46,iface=MIXER,name='ADC OSR Volume ON'
; type=BOOLEAN,access=rw-----,values=2
; values=on,on
root@ATK-DLRV1126B:~# /usr/bin/amixer -c 0 set 'ADC OSR' 100%
Simple mixer control 'ADC OSR',0
Capabilities: volume volume-joined
Playback channels: Mono
Capture channels: Mono
Limits: 0 - 255
Mono: 255 [100%] [32.00dB]
root@ATK-DLRV1126B:~# /usr/bin/amixer -c 0 set 'ADC2DAC Mixer' 90%
Simple mixer control 'ADC2DAC Mixer',0
Capabilities: volume volume-joined
Playback channels: Mono
Capture channels: Mono
Limits: 0 - 127
Mono: 114 [90%] [19.00dB]
root@ATK-DLRV1126B:~#
```

2.4.6 编码录制 MIC 音频

为了能够清晰录音，请先执行 2.4.5 小节的指令再执行下面指令，录制 MIC 音频。按 Ctrl+C 停止。

```
gst-launch-1.0 -e \
    alsasrc device="default" ! \
    audio/x-raw,format=S16LE,rate=48000,channels=2 ! \
    audioconvert ! \
    audioresample ! \
    opusenc bitrate=64000 ! \
    oggmux ! \
    filesink location=test_opus.ogg
```

- `gst-launch-1.0 -e` - GStreamer 命令行工具，`-e` 参数确保文件正确结束。
- `alsasrc device="default"` - 从 ALSA 音频系统默认设备采集音频输入。
- `audio/x-raw,format=S16LE,rate=48000,channels=2` - 设置音频格式为 16 位有符号小端格式，采样率 48kHz，立体声双声道。
- `audioconvert` - 音频格式转换器，确保格式兼容。
- `audioresample` - 音频重采样器，确保采样率一致。
- `opusenc bitrate=64000` - Opus 音频编码器，以 64kbps 比特率编码。
- `oggmux` - OGG 容器复用器，将音频流封装到 OGG 容器中。
- `filesink location=test_opus.ogg` - 文件输出，保存为 `test_opus.ogg` 文件。

执行下面指令编码成 mp3，按 Ctrl+C 停止。

```
gst-launch-1.0 -e \
    alsasrc device="default" ! \
    audio/x-raw,format=S16LE,rate=48000,channels=2 ! \
    audioconvert ! \
```

```
audioresample !\  
lamemp3enc bitrate=128 !\  
filesink location=test.mp3
```

- `gst-launch-1.0 -e` - GStreamer 命令行工具, `-e` 参数确保文件正确结束。
- `alsasrc device="default"` - 从 ALSA 音频系统默认设备采集音频输入。
- `audio/x-raw,format=S16LE,rate=48000,channels=2` - 设置音频格式为 16 位有符号小端格式, 采样率 48kHz, 立体声双声道。
- `audioconvert` - 音频格式转换器, 确保格式兼容。
- `audioresample` - 音频重采样器, 确保采样率一致。
- `lamemp3enc bitrate=128` - LAME MP3 编码器, 以 128kbps 比特率编码。
- `filesink location=test.mp3` - 文件输出, 保存为 test.mp3 文件。

第三章 音视频推流开发

本章将介绍视频推流, 包括 Live555 和 mediamtx 流媒体服务软件的使用, 快速搭建流媒体服务器。简单来说, 流媒体服务软件是将音视频数据以流的形式 (如 RTSP, RTMP, HLS, WebRTC 等) 通过网络进行分发和转发的核心服务端程序。

Live555 它是一个历史悠久、功能核心、偏向底层的 C++ 流媒体库和工具集。带了一些命令行工具 (如 live555MediaServer), 但功能相对简单, 主要用于演示和测试。对 RTSP/RTP/RTCP 的支持是其核心和强项, 被视为该协议的“标准”开源实现, 特点是轻量级。

MediaMTX (原 rtsp-simple-server) 它是一个现代、开箱即用、功能丰富的流媒体服务器, 专门用于实时媒体流转发。协议转换枢纽: 这是它最大的亮点。它能无缝地在多种协议间转换流。输入: 支持 RTSP, RTMP, HLS, HTTP(S), SRT, WebRTC, MPEG-TS 等。输出: 同样支持上述所有协议。无转码: 它只进行容器的解封装和重新封装, 不进行视频/音频的编解码 (转码), 因此延迟极低 (毫秒级), CPU 消耗小。你可以将一个 RTMP 推流 (来自 OBS), 同时转换成 RTSP、HLS 和 WebRTC 流分发出去, 一份输入, 多协议输出。配置驱动: 所有功能 (用户认证、路径映射、协议开关) 通过一个清晰的 YAML 文件配置。简单易用: 单一 Go 语言二进制文件, 无复杂依赖, 部署极其方便。安防/IP 摄像头集成: 将只支持 RTSP 的摄像头, 无延迟地转换为 WebRTC 在浏览器中播放, 或转换为 HLS 在移动端播放。直播流转发: 作为 RTMP/RTSP 的中继服务器, 实现“一推多拉”。构建低延迟的 Web 监控后台。需要同时支持多种播放协议的流媒体应用场景。

在实际应用中, 国产使用 ZLMediaKit 的使用率和知名度确实远超 MediaMTX 和 Live555, 是当前最主流、应用最广泛的开源流媒体服务器之一。其开源仓库为 <https://github.com/ZLMediaKit/ZLMediaKit>。

对于国内用户和开发者:

首选 ZLMediaKit: 如果用户的项目是安防监控 (尤其是需要对接国标平台)、大规模直播、需要高性能和丰富功能 (如转码录制)、或者你希望获得活跃的国内社区支持, ZLMediaKit 是毫无疑问的第一选择。它是经过国内大量商业项目验证的“重型武器”。

考虑 MediaMTX: 如果你的需求非常明确, 就是单纯地、低延迟地在不同协议间转发实时流 (比如把 RTSP 转成 WebRTC 在网页播放), 并且追求极致的简单部署和配置, 不想引入复杂的功能, 那么 MediaMTX 是一个更轻量、更聚焦的优雅工具。它在国外和物联网领域很受欢迎。

使用或学习 Live555: 如果用户是一个学生或研究者, 想深入学习 RTSP/RTP 协议的底层原理, 或者你在维护一个基于 Live555 的旧项目, 它依然有价值。但在开新项目时, 很少会将其作为首选服务器。

当然本章以 Live555 与 MediaMTX 为例, 快速搭建流媒体服务器, 因为它们依赖较少, 容易搭建。实际上流媒体服务软件有很多种, 不限于笔者说的这几种。对于 ZLMediaKit 可以参考 GITHUB 上的文档进行编译构建, 笔者也成功编译并部署 ZLMediaKit。有兴趣的用户可以试试。

3.1 Live555 流媒体软件

正点原子 RV1126B 出厂系统支持 Live555, 输入下面指令可以看到如下内容表示支持 Live555。

```
ls /usr/bin/live555*
```

```
root@ATK-DLRV1126B:/# ls /usr/bin/live555*
/usr/bin/live555HLSProxy      /usr/bin/live555ProxyServer
/usr/bin/live555MediaServer
```

3.1.1 帮助说明

live555MediaServer 的帮助说明如下。

LIVE555 Media Server

version 1.11 (LIVE555 Streaming Media library version 2024.05.15).

Play streams from this server using the URL

rtsp://192.168.6.119/<filename>

where <filename> is a file present in the current directory.

Each file's type is inferred from its name suffix:

".264" => a H.264 Video Elementary Stream file

".265" => a H.265 Video Elementary Stream file

".aac" => an AAC Audio (ADTS format) file

".ac3" => an AC-3 Audio file

".amr" => an AMR Audio file

".dv" => a DV Video file

".m4e" => a MPEG-4 Video Elementary Stream file

".mkv" => a Matroska audio+video+(optional)subtitles file

".mp3" => a MPEG-1 or 2 Audio file

".mpg" => a MPEG-1 or 2 Program Stream (audio+video) file

".ogg" or ".ogv" or ".opus" => an Ogg audio and/or video file

".ts" => a MPEG Transport Stream file

(a ".tsx" index file - if present - provides server 'trick play' support)

".vob" => a VOB (MPEG-2 video with AC-3 audio) file

".wav" => a WAV Audio file

".webm" => a WebM audio(Vorbis)+video(VP8) file

See <http://www.live555.com/mediaServer/> for additional documentation.

(We use port 80 for optional RTSP-over-HTTP tunneling.)

根据上面的解释, live555MediaServer, 会以当前目录下所有如.264/.265...后缀的文件, 当 rtsp://192.168.6.119/<filename> 进行拉流时, 就会播放对应的文件! 简单点来说, live555MediaServer 就是一个点播放机。红色部分是个人开发板 IP, 获取 IP 需要插到路由器上, 插上后自动获取 IP!

3.1.2 live555 推流

由于 live555MediaServer 位于 /usr/bin/ 下, /usr/bin/ 下文件太多, 所以我们需要拷贝一份这个 live555MediaServer 可执行文件到一个文件夹下。

新建一个文件夹 live555, 文件夹随意命名。

```
mkdir /live555
cp /usr/bin/live555MediaServer /live555/
ls /live555/
```

```
root@ATK-DLRV1126B:/# mkdir live555
root@ATK-DLRV1126B:/# cp /usr/bin/live555MediaServer /live555/
root@ATK-DLRV1126B:/# ls live555/
live555MediaServer
root@ATK-DLRV1126B:/#
```

那么 .264/.265 等文件我们去哪里找? 我们已经想到前面第二章有例子怎么编码成 H.264/H.265 文件了。注意了, 我们需要保存后缀为 .264/.265 而不是 .h264/.h265。

鉴于用户可能没有摄像头, 笔者这里先保存一个 GST 产生的视频文件吧。约 10 秒。有摄像头的用户参考第二章的指令保存后缀为 .264/.265 的文件, 这里我们不重复写指令了!

```
cd live555/
gst-launch-1.0 videotestsrc pattern=smpte num-buffers=300 \
! video/x-raw,format=NV12,width=640,height=480 ! \
identity sleep-time=33333 ! mpph264enc ! h264parse ! \
filesink location=test.264
```

视频源配置

每帧延迟 33.333ms = 30fps

videotestsrc pattern=smpte

pattern=smpte: 使用 SMPTE 测试图 (彩条)

可选测试图:

- smpte: 彩条图
- snow: 雪花噪声
- black: 黑屏
- white: 白屏
- red: 红屏

```
root@ATK-DLRV1126B:/live555# cd ..
root@ATK-DLRV1126B:/# cd live555/
root@ATK-DLRV1126B:/live555# gst-launch-1.0 videotestsrc pattern=smpte num-buffers=300 ! video/x-raw,format=NV12,width=640,height=480 ! identity sleep-time=33333 ! mpph264enc !
h264parse ! filesink location=test.264
mpp[1744]: mpp info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1744]: mpp info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1744]: mpp info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1744]: mpp: unable to create enc vp8 for soc rv1126b unsupported
mpp[1744]: mpp info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
Setting pipeline to PAUSED ...
mpp[1744]: mpp info: mpp version: 215fc18 author: alientek 2025-10-20 repository init!
mpp[1744]: mpp: Only rk3588's h264/265/jpeg and rk3576's h264/265 encoder can use frame parallel
Pipeline is PREROLLING ...
mpp[1744]: mpp enc: MPP_ENC_SET_RC_CFG bps 1152000 [1080000 : 1224000] fps [30:30] gop 30
mpp[1744]: h264e api v2: MPP_ENC_SET_PREP_CFG w:h [640:480] stride [640:480]
mpp[1744]: mpp enc: mode cbr bps [1080000:1152000:1224000] fps fix [30/1] -> fix [30/1] gop i [30] v [0]
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:11.844149854
Setting pipeline to NULL ...
Freeing pipeline ...
root@ATK-DLRV1126B:/live555# ls .
live555MediaServer test.264
root@ATK-DLRV1126B:/live555#
```

然后我们直接执行当前目录 live555 下的 live555MediaServer 就可以进行拉流了。如下图。

```
root@ATK-DLRV1126B:/live555# ls
live555MediaServer test.264
root@ATK-DLRV1126B:/live555# ./live555MediaServer
LIVE555 Media Server
    version 1.11 (LIVE555 Streaming Media library version 2024.05.15).
Play streams from this server using the URL
    rtsp://192.168.6.119/<filename>
where <filename> is a file present in the current directory.
Each file's type is inferred from its name suffix:
    ".264" => a H.264 Video Elementary Stream file
    ".265" => a H.265 Video Elementary Stream file
    ".aac" => an AAC Audio (ADTS format) file
    ".ac3" => an AC-3 Audio file
    ".amr" => an AMR Audio file
    ".dv" => a DV Video file
    ".m4e" => a MPEG-4 Video Elementary Stream file
    ".mkv" => a Matroska audio+video+(optional)subtitles file
    ".mp3" => a MPEG-1 or 2 Audio file
    ".mpg" => a MPEG-1 or 2 Program Stream (audio+video) file
    ".ogg" or ".ogv" or ".opus" => an Ogg audio and/or video file
    ".ts" => a MPEG Transport Stream file
        (a ".tsx" index file - if present - provides server 'trick play' support)
    ".vob" => a VOB (MPEG-2 video with AC-3 audio) file
    ".wav" => a WAV Audio file
    ".webm" => a WebM audio(Vorbis)+video(VP8) file
See http://www.live555.com/mediaServer/ for additional documentation.
(We use port 80 for optional RTSP-over-HTTP tunneling).
```

在 Ubuntu 直接使用 GST 进行拉流。若用户 Ubuntu 没有安装 GST 则需要在 Ubuntu 下执行下面指令。这里不建议用 VLC 拉取，原因 VLC 无法调整延时，实时摄像头流延时较大，二有时候打不开流（可能与笔者 VLC 版本有关，笔者不再探究）。

```
sudo apt-get install gstreamer1.0-plugins-base gstreamer1.0-plugins-bad \
gstreamer1.0-libav gstreamer1.0-plugins-good gstreamer1.0-plugins-ugly \
gstreamer1.0-pulseaudio
```

在 Ubuntu 安装上面的指令后执行下面指令进行拉流。效果如下图。播放 10 秒后结束，若要重新拉，再执行一次即可。

```
gst-play-1.0 rtsp://192.168.6.119/test.264
```

```
allientek@allientek:~$ gst-play-1.0 rtsp://192.168.6.119/test.264
按“k”键来显示键盘快捷键列表。
正在播放 rtsp://192.168.6.119/test.264
Pipeline is live.
Redistribute latency...
Redistribute latency...
Prerolled.
```



3.2 MediaMTX 流媒体软件

3.2.1 MediaMTX 简介

MediaMTX 介绍: 单纯地、低延迟地不同协议间转发实时流(比如把 RTSP 转成 WebRTC 在网页播放), 并且追求极致的简单部署和配置, 不想引入复杂的功能, 那么 MediaMTX 是一个更轻量、更聚焦的优雅工具。它在国外和物联网领域很受欢迎。

MediaMTX 软件包, 在 atk_dlv1126b_linux6.1_sdk_release_v1.0.1 版本 SDK 里已经支持, 请看到此文档的用户更新一下 SDK 到 atk_dlv1126b_linux6.1_sdk_release_v1.0.1 版本, 或者从网盘 A 盘 08、系统镜像\01、Linux_buildroot 系统镜像\update.img 烧写此镜像。

检测出厂系统下有没有/usr/bin/mediamtx。执行下面指令。如果没有就从网盘下载最新 A 盘镜像烧写即可。

```
root@ATK-DLRV1126B:/# ls /usr/bin/mediamtx
/usr/bin/mediamtx
root@ATK-DLRV1126B:/#
```

3.2.2 MediaMTX 配置文件

备份文件, 执行下面命令。

```
cp /etc/mediamtx.yml /etc/mediamtx.yml-bak
```

然后将下面内容覆盖到/etc/mediamtx.yml。

```
rtspAddress: ":8554"
rtmpAddress: ":1935"
hlsVariant: fmp4
hlsAddress: ":8888"
webRTCAddress: ":8889"
rtsp: yes
rtmp: yes
webRTC: yes
paths:
  camera:
    sourceOnDemand: no
    disablePublisherOverride: yes
    readUser: viewer
    readPass: viewer123
```

解释:

rtspAddress: ":8554"	# RTSP 协议, 监听 8554 端口
rtmpAddress: ":1935"	# RTMP 协议, 监听 1935 端口 (标准端口)
hlsAddress: ":8888"	# HLS 协议, 监听 8888 端口
webRTCAddress: ":8889"	# WebRTC 协议, 监听 8889 端口
rtsp: yes	# 启用 RTSP 服务
rtmp: yes	# 启用 RTMP 服务
webRTC: yes	# 启用 WebRTC 服务
# 注意: HLS 默认启用, 无需单独开关	
hlsVariant: fmp4	# 使用 fmp4 格式 (现代 HLS, 支持低延迟)
paths:	
camera:	# 流路径名称
sourceOnDemand: no	# 一直保持连接

```
disablePublisherOverride: yes    # 禁止推流覆盖
readUser: viewer                # 播放用户名
readPass: viewer123             # 播放密码
```

更多配置请查看/etc/mediamtx.yml-bak 原文件。配置完成格式如下。

```
rtspAddress: ":8554"
rtmpAddress: ":1935"
hlsVariant: fmp4
hlsAddress: ":8888"
webrtcAddress: ":8889"
rtsp: yes
rtmp: yes
webRTC: yes
paths:
  camera:
    sourceOnDemand: no
    disablePublisherOverride: yes
    readUser: viewer
    readPass: viewer123
```

3.2.3 运行 MediaMTX

因为 MediaMTX 支持多种协议推流，拉流可以用 RTSP/RTMP 拉，也可以用 WebRTC 网页端拉等，由 MediaMTX 自动转换非常方便！

配置完成执行下面指令，将 mediamtx 放后台运行，如下图，运行无其他错误，若有错误，请修正/etc/mediamtx.yml 格式，比如 camera 前面有两个空格。sourceOnDemand 前面有四个空格。

mediamtx /etc/mediamtx.yml &

```
root@ATK-DLRV1126B:/# mediamtx /etc/mediamtx.yml &
[2] 3947
root@ATK-DLRV1126B:/# 2026/02/03 15:05:17 INF MediaMTX v1.7.0
2026/02/03 15:05:17 INF configuration loaded from /etc/mediamtx.yml
2026/02/03 15:05:17 INF [RTSP] listener opened on :8554 (TCP), :8000 (UDP/RTCP), :8001 (UDP/RTCP)
2026/02/03 15:05:17 INF [RTMP] listener opened on :1935
2026/02/03 15:05:17 INF [HLS] listener opened on :8888
2026/02/03 15:05:17 INF [WebRTC] listener opened on :8889 (HTTP), :8189 (ICE/UDP)
2026/02/03 15:05:17 INF [SRT] listener opened on :8890 (UDP)
```

3.2.4 视频 RTSP 推流

开发板 RTSP 推流如下，先测试 640x480 分辨率，IMX415 也可以推流 4K (3840x2160)：

```
gst-launch-1.0 v4l2src device=/dev/video-camera0 \
! video/x-raw,format=NV12,width=640,height=480,framerate=30/1 \
! mpph264enc gop=30 bps=6000000 ! h264parse config-interval=-1 \
! rtspclientsink location=rtsp://127.0.0.1:8554/camera protocols=tcp
```

指令解释：

- **gst-launch-1.0** - GStreamer 命令行工具，用于构建和运行多媒体管道
- **v4l2src** - Video4Linux2 视频采集组件，从摄像头设备获取视频
- **device=/dev/video-camera0** - 指定摄像头设备路径为/dev/video-camera0
- **video/x-raw** - 视频原始数据格式，未压缩的视频帧
- **format=NV12** - 视频格式为 NV12，一种 YUV420 半平面格式
- **width=640,height=480** - 视频分辨率设置为 640×480 像素
- **framerate=30/1** - 视频帧率设置为 30 帧/秒
- **mpph264enc** - Rockchip MPP 硬件 H.264 编码器，使用硬件加速编码

- **gop=30** - 关键帧间隔设置为 30 帧 (约 1 秒一个关键帧)
- **bps=6000000** - 视频比特率设置为 6Mbps (6,000,000 比特/秒)
- **h264parse** - H.264 解析器, 处理 H.264 编码流
- **config-interval=-1** - 参数, 仅在流开始时发送 SPS/PPS 配置信息
- **rtspclientsink** - RTSP 客户端输出组件, 将流推送到 RTSP 服务器
- **location=rtsp://127.0.0.1:8554/camera** - RTSP 服务器地址, 本地 8554 端口, 路径为/camera
- **protocols=tcp** - 参数, 强制使用 TCP 协议传输 (可靠性更高)

Ubuntu 下 RTSP 拉流指令如下: 请填写个人开发板 IP 地址, 局域网内拉, 未优化延时。

```
gst-play-1.0 rtsp://viewer:viewer123@192.168.6.119:8554/camera
```

```
gst-play-1.0 rtsp://192.168.6.119:8554/camera #若无配置播放用户名及密码用这个
```

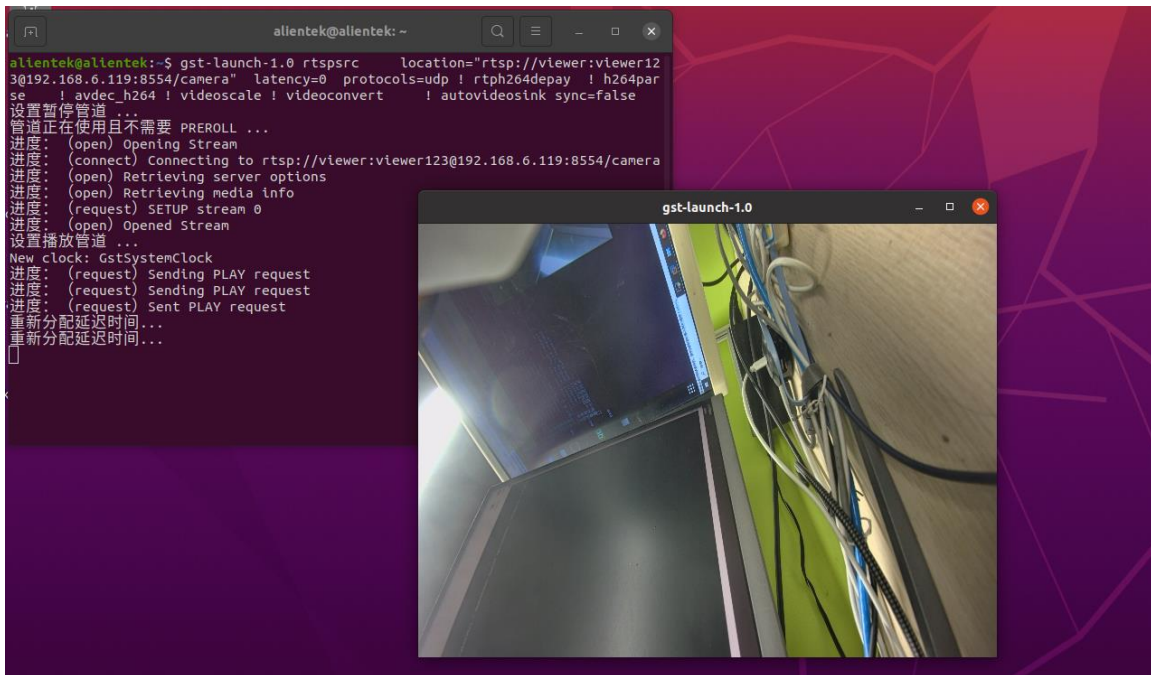
Ubuntu 优化延时如下。

```
gst-launch-1.0 rtspsrc \
  location="rtsp://viewer:viewer123@192.168.6.119:8554/camera" \
  latency=0 protocols=udp ! rtph264depay ! h264parse \
  ! avdec_h264 ! videoscale ! videoconvert \
  ! autovideosink sync=false
```

指令解释:

- **gst-launch-1.0** - GStreamer 命令行工具, 用于构建和运行多媒体管道
- **rtspsrc** - RTSP 客户端源组件, 从 RTSP 服务器拉取流媒体
- **location="rtsp://viewer:viewer123@192.168.6.119:8554/camera"** - RTSP 流地址, 包含用户名密码认证, 服务器 IP 为 192.168.6.119, 端口 8554, 路径/camera
- **latency=0** - 参数, 设置零延迟模式, 最小化缓冲延迟
- **protocols=udp** - 参数, 指定使用 UDP 协议传输数据
- **rtph264depay** - RTP H.264 解包器, 从 RTP 包中提取 H.264 视频数据
- **h264parse** - H.264 解析器, 解析 H.264 视频流的结构和参数
- **avdec_h264** - FFmpeg 的 H.264 软件解码器, 将 H.264 解码为原始视频帧
- **videoscale** - 视频缩放组件, 调整视频帧的尺寸
- **videoconvert** - 视频格式转换器, 在不同色彩空间之间转换
- **autovideosink** - 自动视频渲染器, 根据系统选择最佳显示方式
- **sync=false** - 参数, 禁用同步, 视频以最快速度渲染, 不等待音频

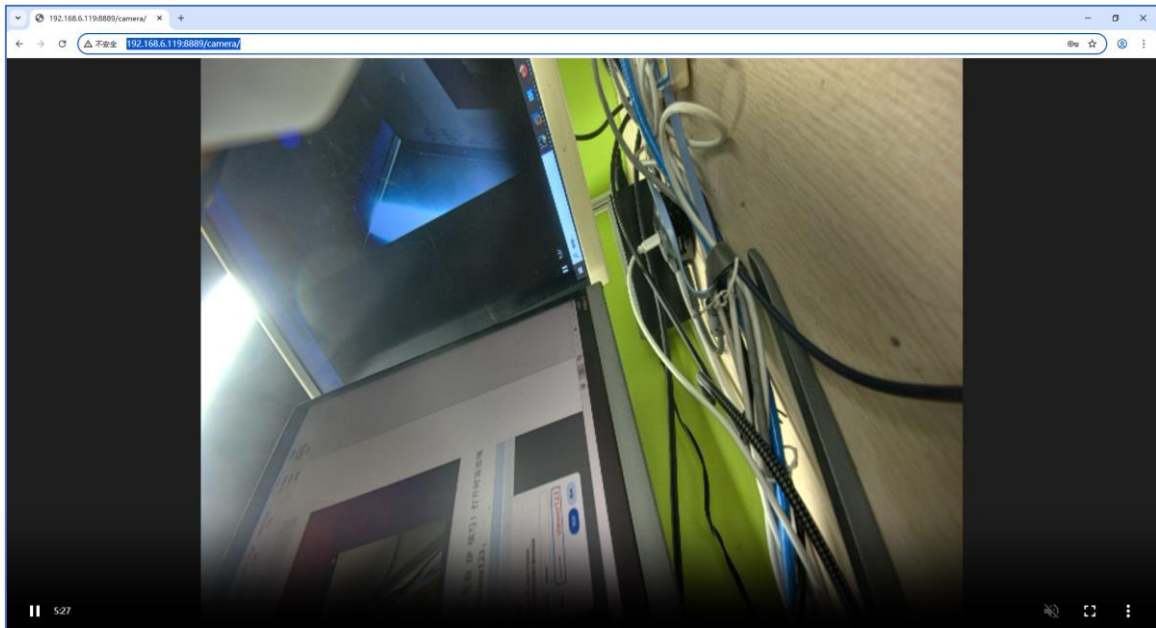
Ubuntu 优化延时运行效果如下图。



MediaMTX 的 8889 端口是 MediaMTX 的 HLS/WebRTC 页面端口。我们需要在浏览器下输入 <http://192.168.6.119:8889/camera/> 根据个人开发板 IP 填写! 打开网页时需要填写用户名及密码, 如下图, 填我们配置的用户名 viewer, 密码 viewer123。



访问 MediaMTX 的 WebRTC 页面, 直接在浏览器中输入: <http://viewer:viewer123@192.168.6.119:8889/camera/>。(根据实际 IP 修改)。打开之后网页端的摄像头画面如下图。



注: WebRTC 不支持 AAC 音频。MediaMTX 的 WebRTC 仅支持 AV1, VP9, VP8, H264, Opus, G722, G711 这几种流格式。加上音频推流的例子请看后面章节。

3.2.5 视频 RTMP 推流

除了 RTSP 协议,也可以使用 RTMP 协议进行推流。RTMP 在直播场景中更为常见,兼容性更好,尤其适合网页播放和社交媒体平台。

在开发板执行下面指令推送到 3.2.3 小节的 Mediamtx 运行的流媒体服务器。

```
gst-launch-1.0 v4l2src device=/dev/video-camera0 \
    ! video/x-raw,format=NV12,width=640,height=480,framerate=30/1 \
    ! mpph264enc gop=30 \
    ! h264parse \
    ! flvmux streamable=true \
    ! rtmpsink location='rtmp://127.0.0.1:1935/camera'
```

指令解释:

- **gst-launch-1.0** - GStreamer 命令行工具,用于构建和运行多媒体管道
- **v4l2src** - Video4Linux2 视频采集组件,从摄像头设备获取视频
- **device=/dev/video-camera0** - 指定摄像头设备路径为/dev/video-camera0
- **video/x-raw** - 视频原始数据格式,表示未压缩的视频帧
- **format=NV12** - 视频像素格式为 NV12,一种 YUV420 半平面格式
- **width=640,height=480** - 视频分辨率设置为 640×480 像素
- **framerate=30/1** - 视频帧率设置为 30 帧/秒
- **mpph264enc** - Rockchip MPP 硬件 H.264 视频编码器,使用硬件加速
- **gop=30** - 关键帧间隔设置为 30 帧(约 1 秒一个关键帧)
- **h264parse** - H.264 解析器,处理 H.264 编码流的结构信息
- **flvmux** - FLV 容器复用器,将视频流封装为 FLV 格式
- **streamable=true** - 参数,设置 FLV 流为可流式传输(适合直播)
- **rtmpsink** - RTMP 输出组件,将流推送到 RTMP 服务器

- **location='rtmp://127.0.0.1:1935/camera'** - RTMP 服务器地址, 本地 1935 端口, 流名称为 camera

Ubuntu 使用 ffplay 拉流, 未优化延时。

```
ffplay 'rtmp://192.168.6.119:1935/camera?user=viewer&pass=viewer123'
```

```
ffplay 'rtmp://192.168.6.119:1935/camera' #无用户及密码时
```

Ubuntu 使用 ffplay 使用 RTMP 拉流, 优化延时。

```
ffplay -fflags nobuffer -flags low_delay \
```

```
'rtmp://192.168.6.119:1935/camera?user=viewer&pass=viewer123'
```

我们也可以使用 RTSP 和 WebRTC 拉流, 请参考 3.2.4 小节的拉流内容。

3.2.6 音视频 RTMP 推流

开发板板载的麦克风(咪头)录音增益默认设置较小, 为确保录音音量充足, 需要先执行以下指令将录音增益调整到合适的较大值。

```
/usr/bin/amixer -c 0 cset name='ADC OSR Volume ON' 'on'
```

```
/usr/bin/amixer -c 0 set 'ADC OSR' 100%
```

```
/usr/bin/amixer -c 0 set 'ADC2DAC Mixer' 90%
```

在开发板执行下面指令推送到 3.2.3 小节的 Mediamtx 运行的流媒体服务器。

```
gst-launch-1.0 -e \
```

```
v4l2src device=/dev/video-camera0 ! \
```

```
video/x-raw,format=NV12,width=640,height=480,framerate=30/1 ! \
```

```
mpph264enc gop=30 ! \
```

```
h264parse config-interval=-1 ! flvmux name=mux ! \
```

```
rtmpsink location=rtmp://127.0.0.1:1935/camera \
```

```
alsasrc device="default" ! \
```

```
audio/x-raw,format=S16LE,rate=48000,channels=2 ! \
```

```
audioconvert ! audioresample ! \
```

```
avenc_aac bitrate=128000 ! \
```

```
aacparse ! mux.
```

指令解释:

- **gst-launch-1.0** - GStreamer 命令行工具, 用于构建和运行多媒体管道
- **-e** - 参数, 在流结束时发送 EOS (End Of Stream) 事件
- **v4l2src** - Video4Linux2 视频采集组件, 从摄像头设备获取视频
- **device=/dev/video-camera0** - 指定摄像头设备路径为/dev/video-camera0
- **video/x-raw** - 视频原始数据格式, 未压缩的视频帧
- **format=NV12** - 视频格式为 NV12, 一种 YUV420 半平面格式
- **width=640,height=480** - 视频分辨率设置为 640×480 像素
- **framerate=30/1** - 视频帧率设置为 30 帧/秒
- **mpph264enc** - Rockchip MPP 硬件 H.264 编码器, 使用硬件加速编码
- **gop=30** - 关键帧间隔设置为 30 帧 (约 1 秒一个关键帧)
- **h264parse** - H.264 解析器, 处理 H.264 编码流
- **config-interval=-1** - 参数, 仅在流开始时发送 SPS/PPS 配置信息
- **flvmux** - FLV 容器复用器, 将音视频流封装为 FLV 格式
- **name=mux** - 将 FLV 复用器命名为"mux", 用于后续连接

- **rtmpsink** - RTMP 输出组件, 将流推送到 RTMP 服务器
- **location=rtmp://127.0.0.1:1935/camera** - RTMP 服务器地址, 本地 1935 端口, 流名称为 camera
- **alsasrc** - ALSA 音频采集组件, 从音频设备获取声音
- **device="default"** - 使用系统默认音频设备
- **audio/x-raw** - 音频原始数据格式, 未压缩的音频采样
- **format=S16LE** - 音频格式为有符号 16 位小端 (Signed 16-bit Little Endian)
- **rate=48000** - 音频采样率为 48kHz
- **channels=2** - 音频声道数为 2 (立体声)
- **audioconvert** - 音频格式转换器, 确保输入格式与编码器兼容
- **audioresample** - 音频重采样器, 确保采样率与编码器要求匹配
- **avenc_aac** - FFmpeg AAC 编码器, 将原始音频编码为 AAC 格式
- **bitrate=128000** - 音频比特率设置为 128kbps
- **aacparse** - AAC 解析器, 处理 AAC 编码流
- **mux.** - 连接到之前命名为 "mux" 的 FLV 复用器的音频输入端口
在 Ubuntu 系统, ffmpeg 使用 RTMP 协议从流媒体服务器拉取音视频流, 优化延时。

```
ffmpeg -fflags nobuffer -flags low_delay \
'rtmp://192.168.6.119:1935/camera?user=viewer&pass=viewer123'
```

Ubuntu 使用 ffmpeg 使用 RTSP 协议从流媒体服务器拉取音视频流, 优化延时。

```
ffmpeg -fflags nobuffer -flags low_delay 'rtsp://viewer:viewer123@192.168.6.119:8554/camera'
```

Ubuntu 使用 RTSP 协议从流媒体服务器拉取音视频流, 指令如下, 有音频和视频。

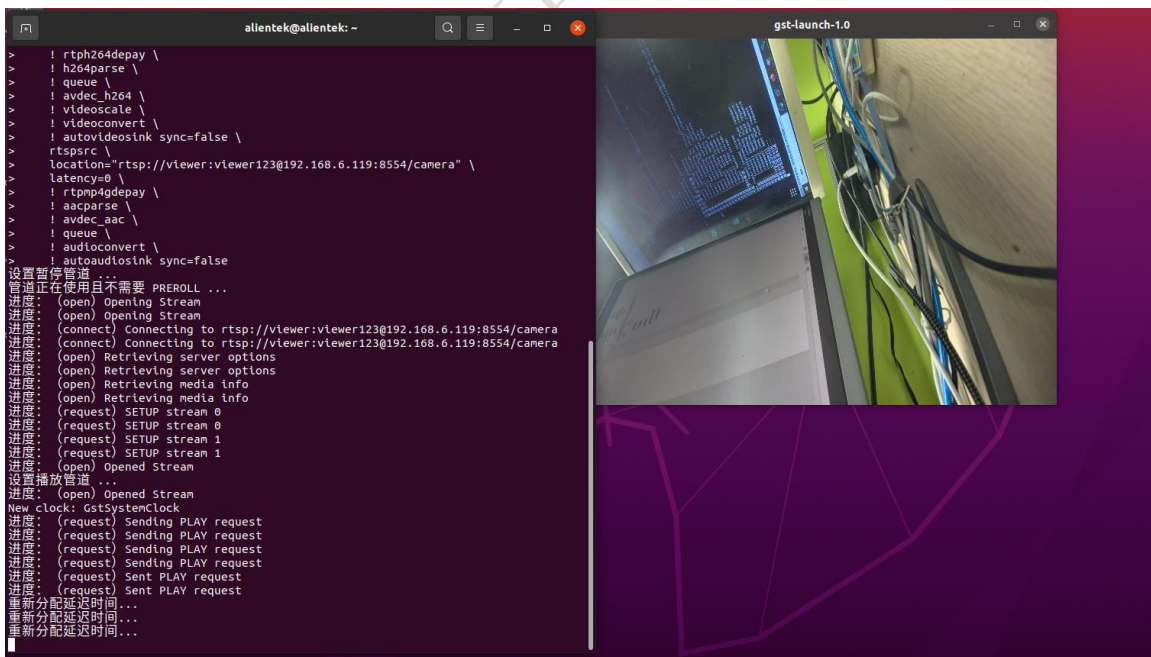
```
gst-launch-1.0 rtspsrc \
    location="rtsp://viewer:viewer123@192.168.6.119:8554/camera" \
    latency=0 \
    protocols=udp \
    ! rtpH264depay \
    ! h264parse \
    ! queue \
    ! avdec_h264 \
    ! videoscale \
    ! videoconvert \
    ! autovideosink sync=false \
    rtspsrc \
    location="rtsp://viewer:viewer123@192.168.6.119:8554/camera" \
    latency=0 \
    ! rtpMP4gdepay \
    ! aacparse \
    ! avdec_aac \
    ! queue \
    ! audioconvert \
    ! autoaudiosink sync=false
```

指令解释:

- **gst-launch-1.0** - GStreamer 命令行工具, 用于构建和运行多媒体管道

- **rtspsrc** - RTSP 客户端源组件, 从 RTSP 服务器拉取流媒体
- **location="rtsp://viewer:viewer123@192.168.6.119:8554/camera"** - RTSP 流地址, 包含用户名密码认证
- **latency=0** - 设置零延迟模式, 最小化缓冲延迟
- **protocols=udp** - 指定使用 UDP 协议传输数据 (默认包含 tcp+udp)
- **rtpH264depay** - RTP H.264 解包器, 从 RTP 包中提取 H.264 视频数据
- **h264parse** - H.264 解析器, 解析 H.264 视频流的结构和参数
- **queue** - 队列缓冲区, 平衡管道中不同组件的处理速度
- **avdec_h264** - FFmpeg 的 H.264 软件解码器, 将 H.264 解码为原始视频帧
- **videoscale** - 视频缩放组件, 调整视频帧的尺寸
- **videoconvert** - 视频格式转换器, 在不同色彩空间之间转换
- **autovideosink** - 自动视频渲染器, 根据系统选择最佳显示方式
- **sync=false** - 禁用同步, 视频以最快速度渲染, 不等待音频
- **rtpmp4gdepay** - RTP MPEG-4 通用解包器, 从 RTP 包中提取 AAC 音频数据
- **aacparse** - AAC 音频解析器, 解析 AAC 音频流的结构
- **avdec_aac** - FFmpeg 的 AAC 软件解码器, 将 AAC 解码为 PCM 音频
- **audioconvert** - 音频格式转换器, 在不同音频格式之间转换
- **autoaudiosink** - 自动音频渲染器, 根据系统选择最佳音频输出设备

同时参考 3.2.4 小节的 WebRTC 在网页端拉流, 网页没有声音的。上面指令推流的是 AAC 音频, 由于 WebRTC 规范不包含 AAC 解码器, 因此无法播放音频。MediaMTX 的 WebRTC 仅支持 AV1, VP9, VP8, H264, Opus, G722, G711 这几种流格式。



3.2.7 音视频 RTSP 推流

同理也要开启录音增益, 看 3.2.6 小节, 先把录音增益开启。

音频视频 RTSP 推流流程如下。

视频源 → 视频处理 → 队列 → 多路复用器(.)

音频源 → 音频处理 → 队列 → 多路复用器(.)

多路复用器 → RTSP 客户端输出

在开发板执行下面指令推送到 3.2.3 小节的 Mediamtx 运行的流媒体服务器。

```
gst-launch-1.0 -e \
v4l2src device=/dev/video-camera0 !\
video/x-raw,format=NV12,width=640,height=480,framerate=30/1 !\
mpph264enc gop=30 !\
h264parse config-interval=-1 !\
queue ! mux.\
alsasrc device="default" !\
audio/x-raw,format=S16LE,rate=48000,channels=2 !\
audioconvert ! audioresample !\
avenc_aac bitrate=128000 !\
aacparse !\
queue ! mux.\
rtspclientsink name=mux location=rtsp://127.0.0.1:8554/camera \
protocols=tcp latency=0
```

指令解释:

- **gst-launch-1.0** - GStreamer 命令行工具, 用于构建和运行多媒体管道
- **-e** - 参数, 在流结束时发送 EOS (End Of Stream) 事件
- **v4l2src** - Video4Linux2 视频采集组件, 从摄像头设备获取视频
- **device=/dev/video-camera0** - 指定摄像头设备路径为 /dev/video-camera0
- **video/x-raw** - 视频原始数据格式, 未压缩的视频帧
- **format=NV12** - 视频格式为 NV12, 一种 YUV420 半平面格式
- **width=640,height=480** - 视频分辨率设置为 640×480 像素
- **framerate=30/1** - 视频帧率设置为 30 帧/秒
- **mpph264enc** - Rockchip MPP 硬件 H.264 编码器, 使用硬件加速编码
- **gop=30** - 关键帧间隔设置为 30 帧 (约 1 秒一个关键帧)
- **h264parse** - H.264 解析器, 处理 H.264 编码流
- **config-interval=-1** - 参数, 仅在流开始时发送 SPS/PPS 配置信息
- **queue** - 队列缓冲区, 平衡管道中不同组件的处理速度
- **mux.** - 连接到名为 "mux" 的复用器的视频输入端口
- **alsasrc** - ALSA 音频采集组件, 从音频设备获取声音
- **device="default"** - 使用系统默认音频设备
- **audio/x-raw** - 音频原始数据格式, 未压缩的音频采样
- **format=S16LE** - 音频格式为有符号 16 位小端 (Signed 16-bit Little Endian)
- **rate=48000** - 音频采样率为 48kHz
- **channels=2** - 音频声道数为 2 (立体声)
- **audioconvert** - 音频格式转换器, 确保输入格式与编码器兼容
- **audioresample** - 音频重采样器, 确保采样率与编码器要求匹配
- **avenc_aac** - FFmpeg AAC 编码器, 将原始音频编码为 AAC 格式
- **bitrate=128000** - 音频比特率设置为 128kbps
- **aacparse** - AAC 解析器, 处理 AAC 编码流

- **rtspclientsink** - RTSP 客户端输出组件, 将流推送到 RTSP 服务器
- **name=mux** - 将 RTSP 输出组件命名为"mux", 作为复用器使用
- **location=rtsp://127.0.0.1:8554/camera** - RTSP 服务器地址, 本地 8554 端口, 路径为 /camera
- **protocols=tcp** - 参数, 强制使用 TCP 协议传输 (可靠性更高)
- **latency=0** - 参数, 设置零延迟模式, 最小化推流延迟

Ubuntu 拉流同 3.2.6 小节, 音频轨道推流的是 AAC, 由于 WebRTC 规范不包含 AAC 解码器, 因此无法播放音频。

3.2.8 WebRTC 音视频

WebRTC (Web Real-Time Communication) 是一个开源项目和技术标准, 它提供了一套 API 和协议, 让浏览器和移动应用能够进行实时音视频通信。

在前面的 3.2.6 小节和 3.2.7 小节, 由于推流的音频编码是 AAC, 所以在网页上 AAC 编码的音频不被 WebRTC 支持。有几个关键原因: WebRTC 标准制定: WebRTC 标准由 IETF 和 W3C 共同制定初始编码选择; 早期选择了 Opus 作为默认音频编码; 专利问题: AAC 有专利授权问题, Opus 是开源免专利的。

在技术特性对比方面, AAC 和 Opus 编码在多个关键指标上存在显著差异, 这些差异直接影响了它们在 WebRTC 场景中的适用性。

延迟表现上, AAC 编码的延迟相对较高, 通常在 20 到 100 毫秒之间, 这种延迟水平对于需要实时交互的语音通话来说可能造成明显的对话不同步感。相比之下, Opus 编码的延迟极低, 范围在 5 到 65 毫秒之间, 能够满足实时通信中对即时响应的严格要求。

带宽适应性方面, AAC 编码通常采用固定比特率模式, 缺乏根据网络条件动态调整的能力。而 Opus 编码展现出卓越的带宽自适应特性, 支持从 6kbps 到 510kbps 的宽范围动态调整, 能够根据实时网络状况智能调节编码参数, 确保在各种网络环境下都能保持最佳的通话质量。

错误恢复机制是另一个关键区别。AAC 编码的错误恢复能力相对较弱, 在数据包丢失时容易出现音频中断或质量下降。Opus 编码则集成了先进的前向纠错(FEC)和数据包丢失隐藏(PLC)技术, 这些技术专门针对不可靠的网络传输环境设计, 能够有效应对网络波动和数据包丢失问题。

语音优化程度也不同, AAC 编码作为通用音频编码标准, 对语音信号的优化处理较为一般。Opus 编码则专门针对语音通信场景进行了深度优化, 在语音清晰度和可懂度方面表现更为出色。虽然 AAC 在音乐内容的编码质量上表现优秀, 但这一特性在主要以语音传输为核心的 WebRTC 应用场景中属于次要考虑因素。

由于 <http://192.168.6.119:8889/camera/> 这个 Mediamtx 的 WebRTC 页面 (8889 端口) 只是一个单向的播放器, 只能看/听, 不能互相通信, 或者需要我们实现完整 WebRTC 逻辑。如果我们要实现双向通信, 建议继续学习 ZLMediaKit, ZLMediaKit 提供双向通信示例代码。github 网址为 <https://github.com/ZLMediaKit/ZLMediaKit>。

下面我们来实现 Mediamtx 的 WebRTC 网页页面接收音频。

在前面几节我们知道 WebRTC 标准不支持 AAC 音频编码。当 MediaMTX 将包含 AAC 音频的流转发给 WebRTC 客户端时, 浏览器能识别到音频流的存在, 但由于 WebRTC 规范不包含 AAC 解码器, 因此无法播放音频。

在开发板执行下面指令推送到 3.2.3 小节的 Mediamtx 运行的流媒体服务器。

```
gst-launch-1.0 -e \
v4l2src device=/dev/video-camera0 !\

```

```
video/x-raw,format=NV12,width=640,height=480,framerate=30/1!\
mpph264enc gop=30!\
h264parse config-interval=-1!\
queue! mux.\
alsasrc device="default"!\
audio/x-raw,format=S16LE,rate=48000,channels=2!\
audioconvert! audioresample!\
opusenc bitrate=64000!\
queue! mux.\
rtspclientsink name=mux location=rtsp://127.0.0.1:8554/camera\
protocols=tcp latency=0
```

指令解释:

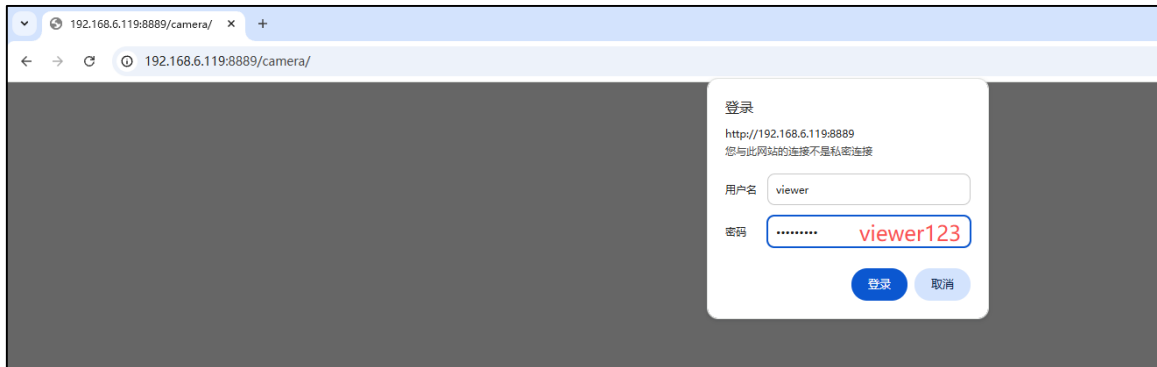
- **gst-launch-1.0** - GStreamer 命令行工具, 用于构建和运行多媒体管道
 - **-e** - 参数, 在流结束时发送 EOS (End Of Stream) 事件
 - **v4l2src** - Video4Linux2 视频采集组件, 从摄像头设备获取视频
 - **device=/dev/video-camera0** - 指定摄像头设备路径为/dev/video-camera0
 - **video/x-raw** - 视频原始数据格式, 未压缩的视频帧
 - **format=NV12** - 视频像素格式为 NV12, 一种 YUV420 半平面格式
 - **width=640,height=480** - 视频分辨率设置为 640×480 像素
 - **framerate=30/1** - 视频帧率设置为 30 帧/秒
 - **mpph264enc** - Rockchip MPP 硬件 H.264 编码器, 使用硬件加速编码
 - **gop=30** - 关键帧间隔设置为 30 帧 (约 1 秒一个关键帧)
 - **h264parse** - H.264 解析器, 处理 H.264 编码流
 - **config-interval=-1** - 参数, 仅在流开始时发送 SPS/PPS 配置信息
 - **queue** - 队列缓冲区, 平衡管道中不同组件的处理速度
 - **mux.** - 连接到名为"mux"的复用器的视频输入端口
 - **alsasrc** - ALSA 音频采集组件, 从音频设备获取声音
 - **device="default"** - 使用系统默认音频设备
 - **audio/x-raw** - 音频原始数据格式, 未压缩的音频采样
 - **format=S16LE** - 音频格式为有符号 16 位小端 (Signed 16-bit Little Endian)
 - **rate=48000** - 音频采样率为 48kHz
 - **channels=2** - 音频声道数为 2 (立体声)
 - **audioconvert** - 音频格式转换器, 确保输入格式与编码器兼容
 - **audioresample** - 音频重采样器, 确保采样率与编码器要求匹配
 - **opusenc** - Opus 音频编码器, 将原始音频编码为 Opus 格式
 - **bitrate=64000** - Opus 编码比特率设置为 64kbps
 - **rtspclientsink** - RTSP 客户端输出组件, 将流推送到 RTSP 服务器
 - **name=mux** - 将 RTSP 输出组件命名为"mux", 作为复用器使用
 - **location=rtsp://127.0.0.1:8554/camera** - RTSP 服务器地址, 本地 8554 端口, 路径为 /camera
 - **protocols=tcp** - 参数, 强制使用 TCP 协议传输 (可靠性更高)
 - **latency=0** - 参数, 设置零延迟模式, 最小化推流延迟
- 在 Ubuntu 系统, ffplay 使用 RTSP 协议从流媒体服务器拉取音视频流, 优化延时。

ffmpeg -fflags nobuffer-flags low_delay 'rtsp://viewer:viewer123@192.168.6.119:8554/camera'
Ubuntu 使用 RTSP 协议从流媒体服务器拉取音视频流, 指令如下, 有音频和视频。

```
gst-launch-1.0 rtspsrc \  
    location="rtsp://viewer:viewer123@192.168.6.119:8554/camera" \  
    latency=0 \  
    protocols=udp \  
    ! rtpH264depay \  
    ! h264parse \  
    ! queue \  
    ! avdec_h264 \  
    ! videoscale \  
    ! videoconvert \  
    ! autovideosink sync=false \  
rtspsrc \  
    location="rtsp://viewer:viewer123@192.168.6.119:8554/camera" \  
    latency=0 \  
    ! rtpOpusdepay \  
    ! opusdec \  
    ! queue \  
    ! audioconvert \  
    ! autoaudiosink sync=false
```

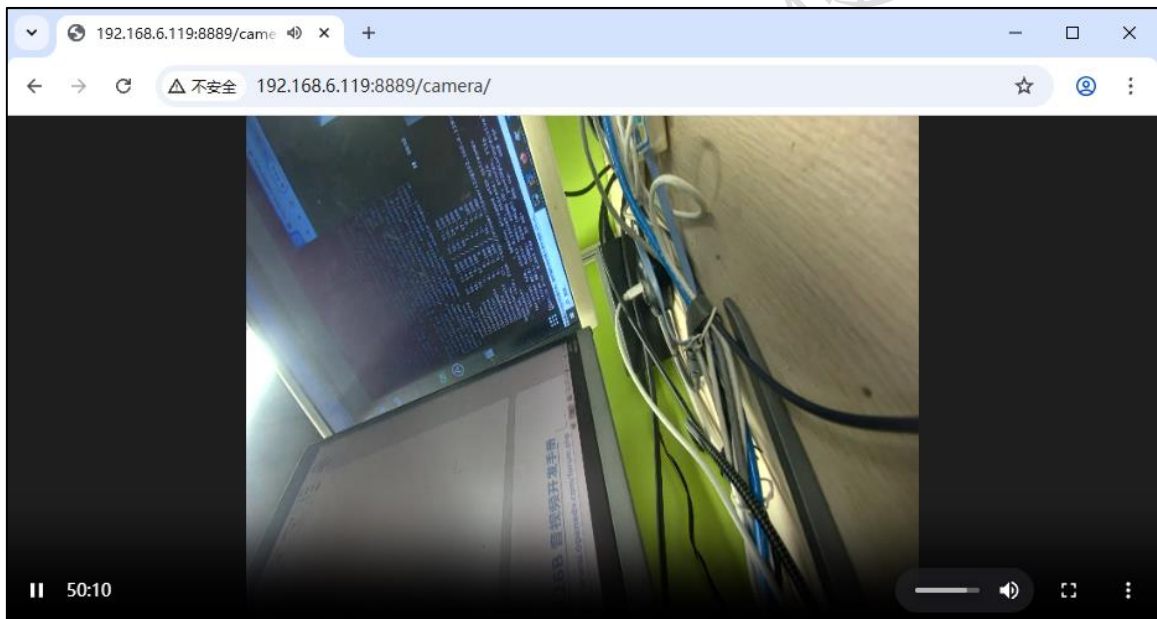
- **gst-launch-1.0** - GStreamer 命令行工具, 用于构建和运行多媒体管道
- **rtspsrc** - RTSP 客户端源组件, 从 RTSP 服务器获取流媒体
- **location="rtsp://viewer:viewer123@192.168.6.119:8554/camera"** - RTSP 流地址, 包含认证信息: 用户名为 viewer, 密码为 viewer123, 服务器地址 192.168.6.119, 端口 8554, 路径 /camera
- **latency=0** - 参数, 设置零延迟模式, 最小化缓冲延迟
- **protocols=udp** - 参数, 指定使用 UDP 协议传输数据
- **rtpH264depay** - RTP H.264 解包器, 从 RTP 数据包中提取 H.264 视频数据
- **h264parse** - H.264 解析器, 解析 H.264 视频流的结构和参数
- **queue** - 队列缓冲区, 平衡管道中不同组件的处理速度
- **avdec_h264** - FFmpeg 的 H.264 软件解码器, 将 H.264 解码为原始视频帧
- **videoscale** - 视频缩放组件, 调整视频帧的尺寸
- **videoconvert** - 视频格式转换器, 在不同色彩空间之间转换
- **autovideosink** - 自动视频渲染器, 根据系统选择最佳显示方式
- **sync=false** - 参数, 禁用视频同步, 以最快速度渲染
- **rtpOpusdepay** - RTP Opus 解包器, 从 RTP 包中提取 Opus 音频数据
- **opusdec** - Opus 解码器, 将 Opus 编码解码为 PCM 音频
- **audioconvert** - 音频格式转换器, 在不同音频格式之间转换
- **autoaudiosink** - 自动音频渲染器, 根据系统选择最佳音频输出设备

MediaMTX 的 8889 端口是 MediaMTX 的 HLS/WebRTC 页面端口。我们需要在浏览器下输入 <http://192.168.6.119:8889/camera/> 根据个人开发板 IP 填写! 打开网页时需要填写用户名及密码, 如下图, 填我们配置的用户名 viewer, 密码 viewer123。



访问 MediaMTX 的 WebRTC 页面, 直接在浏览器中输入: <http://viewer:viewer123@192.168.6.119:8889/camera/>。(根据实际 IP 修改)。

打开之后网页端的摄像头画面如下图。右下角有个音量标记, 注意, 不同的浏览器可能显示界面不一样, 下图是笔者使用的是 Google Chrome 显示的。



第四章 GStreamer 的音视频推流

本章使用 GStreamer 框架编写应用程序, 实现音视频同步采集、编码, 并通过 RTSP 协议进行实时推流。

仅供正点原子客户使用

4.1 GStreamer 音视频 RTSP 推流

4.1.1 程序源码及编译

音视频 RTSP 推流 GStreamer 源码下分为 build.sh、CMakeLists.txt 和 gst-rtsp-server.c 源程序。源码在正点原子用户 RV1126B 开发板用户群文件《正点原子客户专用资料》音视频 RTSP 推流 GStreamer 源码.rar。



解压音视频 RTSP 推流 GStreamer 源码.rar 得到文件如下。

名称	修改日期	类型	大小
build.sh	2026/2/3 18:25	Shell Script	1 KB
CMakeLists.txt	2026/2/3 19:20	文本文档	2 KB
gst-rtsp-server	2026/2/3 19:19	文件	36 KB
gst-rtsp-server.c	2026/2/3 19:09	C Source file	14 KB

拷贝源码拷贝到 Ubuntu, 根据 09、用户手册\02、辅助文档|【正点原子】基于 Buildroot 系统_交叉编译器安装与使用参考手册, 先安装工具链。

如下图, 拷贝到新建的 gst-rtsp-server 文件夹下。

```

alientek@alientek:~/gst-rtsp-server$ chmod +x build.sh
alientek@alientek:~/gst-rtsp-server$ ls
build.sh  CMakeLists.txt  gst-rtsp-server.c
alientek@alientek:~/gst-rtsp-server$

alientek@alientek:~/gst-rtsp-server$ ./build.sh
作者: dengzhinao@alientek.com
B站: dengzhinao
license  LGPL-2.1-or-later

Buildroot
Making embedded Linux easy!

Some tips:
* PATH now contains the SDK utilities
* Standard autotools variables (CC, LD, CFLAGS) are exported
* Kernel compilation variables (ARCH, CROSS_COMPILE, KERNELDIR) are exported
* To configure do "./configure $CONFIGURE_FLAGS" or use the "configure" alias
* To build CMake-based projects, use the "cnae" alias

-- cnae version 3.28.1
-- The C compiler identification is GNU 13.3.0
-- The CXX compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /opt/atk-dlrv1126b-toolchain/bin/aarch64-buildroot-linux-gnu-gcc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /opt/atk-dlrv1126b-toolchain/bin/aarch64-buildroot-linux-gnu-g++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- cnae version 3.28.1
-- Found PkgConfig: /opt/atk-dlrv1126b-toolchain/bin/pkg-config (found version "1.6.3")
-- Checking for modules 'gstreamer-1.0;gstreamer-rtsp-server-1.0'
-- Found gstreamer-1.0, version 1.24.11
-- Found gstreamer-rtsp-server-1.0, version 1.24.11
-- Configuring done (0.5s)
-- Generating done (0.0s)
-- Build files have been written to: /home/alientek/gst-rtsp-server
[ 50%] Building C object CMakeFiles/gst-rtsp-server.dir/gst-rtsp-server.c.o
[100%] Linking C executable gst-rtsp-server
[100%] Built target gst-rtsp-server
alientek@alientek:~/gst-rtsp-server$
    
```

编译成功后, 在当前路径下拷贝 `gst-rtsp-server` 到开发板。

```
alientek@alientek:~/gst-rtsp-server$ ls
build.sh  CMakeCache.txt  CMakeFiles  cmake_install.cmake  CMakeLists.txt  gst-rtsp-server  gst-rtsp-server.c  Makefile
alientek@alientek:~/gst-rtsp-server$
```

本次使用 `adb` 传送到开发板/路径下。

```
alientek@alientek:~/gst-rtsp-server$ adb push gst-rtsp-server /
gst-rtsp-server: 1 file pushed. 1.4 MB/s (36568 bytes in 0.025s)
alientek@alientek:~/gst-rtsp-server$
```

4.1.2 帮助说明

在开发板路径下执行下面指令, 查看帮助。

`./gst-rtsp-server --help`

```
root@ATK-DLRV1126B:/# ./gst-rtsp-server --help
=====
RTSP Audio/Video Server v2.0
=====
RTSP Audio/Video Stream Server
Usage: ./gst-rtsp-server [options]

Video Configuration:
-d, --device PATH      Camera device path (default: /dev/video-camera0)
-f, --format FORMAT    Video format (default: NV12)
-w, --width WIDTH      Video width (default: 640)
-h, --height HEIGHT    Video height (default: 480)
-r, --fps FPS          Frame rate (default: 30)

Audio Configuration:
-a, --audio-device DEVICE Audio device (default: default)
                        Options: default, pulser, hw:0,0, etc.
-s, --sample-rate RATE  Sample rate (default: 44100)
-c, --channels CHANNELS Channels 1/2 (default: 2)
-q, --quality QUALITY   Vorbis quality 0-1 (default: 0.7)
--no-audio              Disable audio streaming

Server Configuration:
-p, --port PORT         RTSP port (default: 8554)
-m, --mount PATH        Mount path (default: /stream)
--help                 Show this help message

Examples:
./gst-rtsp-server -d /dev/video0 -a default      (audio+video)
./gst-rtsp-server --no-audio                    (video only)
./gst-rtsp-server -d /dev/video1 -f NV12 -a pulser (custom config)
root@ATK-DLRV1126B:/#
```

4.1.3 设置录音增益

开发板板载的麦克风（咪头）录音增益默认设置较小, 为确保录音音量充足, 需要先执行以下指令将录音增益调整到合适的较大值。

```
/usr/bin/amixer -c 0 cset name='ADC OSR Volume ON' 'on'
/usr/bin/amixer -c 0 set 'ADC OSR' 100%
/usr/bin/amixer -c 0 set 'ADC2DAC Mixer' 90%
```

```
root@ATK-DLRV1126B:/# /usr/bin/amixer -c 0 cset name='ADC OSR Volume ON' 'on'
numid=46,iface=MIXER,name='ADC OSR Volume ON'
; type=BOOLEAN,access=rw-----,values=2
; values=on,on
root@ATK-DLRV1126B:/# /usr/bin/amixer -c 0 set 'ADC OSR' 100%
Simple mixer control 'ADC OSR',0
Capabilities: volume volume-joined
Playback channels: Mono
Capture channels: Mono
Limits: 0 - 255
Mono: 255 [100%] [32.00dB]
root@ATK-DLRV1126B:/# /usr/bin/amixer -c 0 set 'ADC2DAC Mixer' 90%
Simple mixer control 'ADC2DAC Mixer',0
Capabilities: volume volume-joined
Playback channels: Mono
Capture channels: Mono
Limits: 0 - 127
Mono: 114 [90%] [19.00dB]
root@ATK-DLRV1126B:/#
```

4.1.4 RTSP 推流

执行下面指令指定 MIPI 摄像头设备与音频设备。

```
./gst-rtsp-server -d /dev/video-camera0 -a default
```

```
root@ATK-DLKV1126B:~# ./gst-rtsp-server -d /dev/video-camera0 -a default
=====
RTSP Audio/Video Server v2.0
=====
Audio device: default
Audio configuration: 44100Hz, 2 channels, quality=0.70

GStreamer Pipeline:
( v4l2src device=/dev/video-camera0 ! video/x-raw,format=NV12,width=640,height=480,framerate=30/1 ! mpph264enc ! rtpH264pay name=pay0 pt=96 ! alsasrc device=default ! audio/x-raw,format=S16LE,rate=44100,channels=2 ! audioconvert ! audiosample ! vorbisenc quality=0.70 ! rtpvorbispay name=pay1 pt=97 )

Server Configuration:
Video: /dev/video-camera0 @ 640x480 30fps NV12
Audio: default @ 44100Hz 2ch (Vorbis quality=0.70)
RTSP URL: rtsp://-IP-:8554/stream

To connect from another device, use:
rtsp://192.168.6.119:8554/stream
Local: rtsp://localhost:8554/stream

Press Ctrl+C to stop the server
=====
Server started successfully!
```

注：默认摄像头设备为/dev/video-camera0，音频设备为 default，程序配置了默认摄像头分辨率与采集格式，音频默认配置了通道及采样率，详细请看源码。

4.1.5 RTSP 拉流

Ubuntu 使用 RTSP 协议从流媒体服务器拉取音视频流，指令如下，有音频和视频。

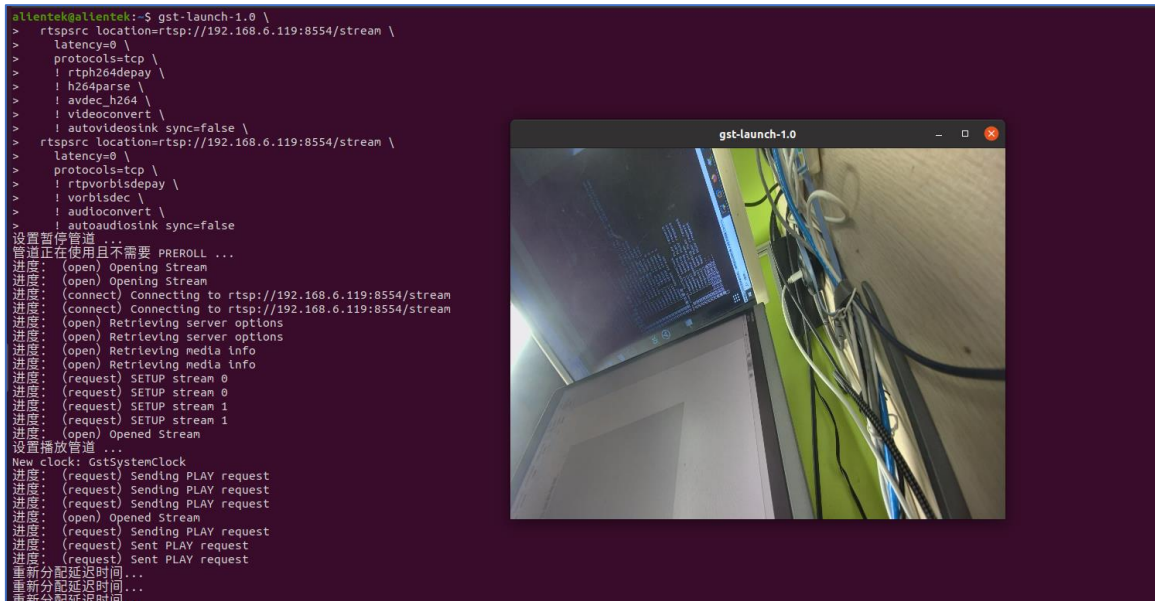
```
gst-launch-1.0 \
    rtspsrc location=rtsp://192.168.6.119:8554/stream \
    latency=0 \
    protocols=tcp \
    ! rtpH264depay \
    ! h264parse \
    ! avdec_h264 \
    ! videoconvert \
    ! autovideosink sync=false \
    rtspsrc location=rtsp://192.168.6.119:8554/stream \
    latency=0 \
    protocols=tcp \
    ! rtpvorbisdepay \
    ! vorbisdec \
    ! audioconvert \
    ! autoaudiosink sync=false
```

指令解释：

- **gst-launch-1.0** - GStreamer 命令行工具，用于构建和运行多媒体管道
- **rtspsrc** - RTSP 客户端源组件，从 RTSP 服务器获取流媒体
- **location=rtsp://192.168.6.119:8554/stream** - RTSP 流地址，服务器 IP 为 192.168.6.119，端口 8554，路径/stream
- **latency=0** - 参数，设置零延迟模式，最小化缓冲延迟
- **protocols=tcp** - 参数，指定使用 TCP 协议传输数据（可靠但延迟稍高）
- **rtpH264depay** - RTP H.264 解包器，从 RTP 数据包中提取 H.264 视频数据
- **h264parse** - H.264 解析器，解析 H.264 视频流的结构和参数
- **avdec_h264** - FFmpeg 的 H.264 软件解码器，将 H.264 解码为原始视频帧
- **videoconvert** - 视频格式转换器，在不同色彩空间之间转换

- **autovideosink** - 自动视频渲染器, 根据系统选择最佳显示方式
- **sync=false** - 参数, 禁用视频同步, 以最快速度渲染
- **rtpvorbisdepay** - RTP Vorbis 解包器, 从 RTP 包中提取 Vorbis 音频数据
- **vorbisdec** - Vorbis 解码器, 将 Vorbis 编码解码为 PCM 音频
- **audioconvert** - 音频格式转换器, 在不同音频格式之间转换
- **autoaudiosink** - 自动音频渲染器, 根据系统选择最佳音频输出设备

Ubuntu 拉流画面如下。



```

allentek@allentek:~$ gst-launch-1.0 \
> rtspsrc location=rtsp://192.168.6.119:8554/stream \
> latency=0 \
> protocols=tcp \
> ! rtp264depay \
> ! h264parse \
> ! avdec_h264 \
> ! videoconvert \
> ! autovideosink sync=false \
> rtspsrc location=rtsp://192.168.6.119:8554/stream \
> latency=0 \
> protocols=tcp \
> ! rtpvorbisdepay \
> ! vorbisdec \
> ! audioconvert \
> ! autoaudiosink sync=false
设置暂停管道 ...
管道正在使用且不需要 PREROLL ...
进度: (open) Opening Stream
进度: (open) Opening Stream
(connect) Connecting to rtsp://192.168.6.119:8554/stream
进度: (open) Retrieving server options
进度: (open) Retrieving server options
进度: (open) Retrieving media info
进度: (open) Retrieving media info
进度: (request) SETUP stream 0
进度: (request) SETUP stream 0
进度: (request) SETUP stream 1
进度: (request) SETUP stream 1
进度: (open) Opened Stream
设置播放管道 ...
New clock: GstSystemClock
进度: (request) Sending PLAY request
进度: (request) Sending PLAY request
进度: (request) Sending PLAY request
进度: (open) Opened Stream
进度: (request) Sending PLAY request
进度: (request) Sent PLAY request
进度: (request) Sent PLAY request
重新分配延迟时间...
重新分配延迟时间...
重新分配延迟时间...

```